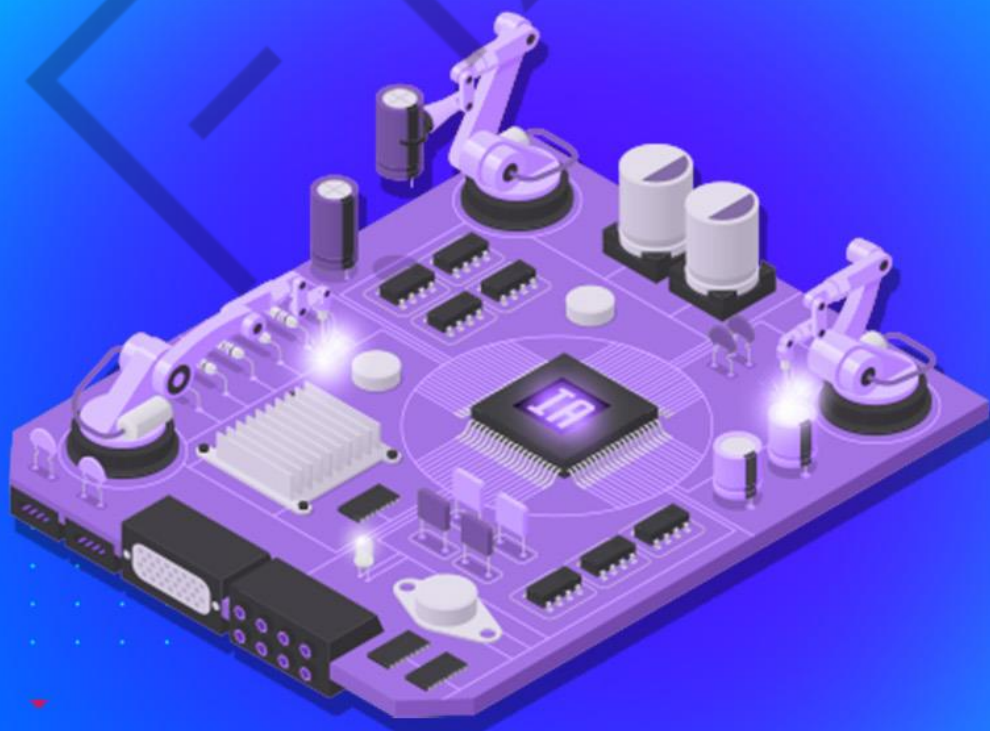


△
COLHEITA DE SOLUÇÕES
INTELIGENTES - TRANSFORMANDO
DADOS EM AÇÕES

ESP32: A JANELA PARA A VISÃO COMPUTACIONAL



3

LISTA DE FIGURAS

Figura 1 - ESP32 com uma pequena câmera (OV2640) de 2MP	7
Figura 2 - Raspberry Pi 4, considerando o menor PC do mundo	8
Figura 3 - Jetson Nano, da NVIDIA	9
Figura 4 - ESP32-CAM com câmera OV2640 da AI Thinker	10
Figura 5 - Placa da empresa FTDI com conector micro USB	10
Figura 6 - Kit com 40 jumpers fêmea-fêmea coloridos	11
Figura 7 - Tipos de cabos USB. À esquerda, mini USB, ao centro, USB-C, à direita, micro USB	11
Figura 8 - Saboneteira plástica simples de qualquer cor	12
Figura 9 - ESP32 CAM à esquerda com a placa FTDI (vermelha) à direita	14
Figura 10 - Fluxograma das etapas do projeto reconhecedor de objetos	17
Figura 11 - Arquitetura do projeto e suas tecnologias	18
Figura 12 - Tela do Visual Studio indicando a instalação do Python	19
Figura 13 - Imagem do ambiente do VS indicando o que está instalado	19
Figura 14 - Barra de progresso de instalação do Arduino IDE	20
Figura 15 - Tela de conclusão da instalação	20
Figura 16 - Exemplo de tela de segurança do Windows solicitando permissão	21
Figura 17 - Cliques e preenchimentos que devem ser feitos para encontrar a família ESP32	21
Figura 18 - Download da família ESP32 sendo realizado	22
Figura 19 - Dica para saber se a família de placas ESP32 foi instalada corretamente	22
Figura 20 - Conferindo se a placa ESP32-CAM foi instalada corretamente	23
Figura 21 - Encontrando a plaquinha ESP32-CAM da AI Thinker	24
Figura 22 - Ambiente do Google Drive com as pastas e subpastas	27
Figura 23 - Povoando a subpasta de treino com X imagens	28
Figura 24 - Imagem com 2 objetos, boneco e um telhado	29
Figura 25 - Site do Make Sense. Pode haver alguma atualização do layout sem aviso prévio	29
Figura 26 - Definindo os rótulos das imagens	30
Figura 27 - Criando as áreas limitantes dos objetos e associando ao seu label	31
Figura 28 - Exportando as coordenadas XY delimitadoras das imagens	31
Figura 29 - Exportando no padrão do YOLO	32
Figura 30 - Criação da pasta labels e subpastas treino e teste no Google Drive	33
Figura 31 - Criando um Colab novo e células de código	34
Figura 32 - Instanciando um computador mais poderoso no Colab, isto é, GPU T4	35
Figura 33 - Local do arquivo telhado.yaml no seu Google Drive. Mesmo nível das subpastas	36
Figura 34 - Caixas delimitadores reconhecendo cada objeto do cenário	39
Figura 35 - Detecção do boneco com 61% de acurácia	42
Figura 36 - ESP32-CAM conectado à interface USB da fabricante FTDI	43
Figura 37 - Montagem real do ESP32-CAM	44
Figura 38 - Ligações eletrônicas detalhadas	45
Figura 39 - Gerenciador de Dispositivos indicando o #USB da sua plaquinha	46
Figura 40 - Site da Silicon Labs, fabricante do chip USB disponível na placa FTDI	46
Figura 41 - Tela do Arduino IDE indicando onde colar o código-fonte fornecido	49
Figura 42 - Tela do repositório Git indicando como fazer download do *.zip	50

Figura 43 - Inclusão manual da biblioteca ZIP do GitHub	50
Figura 44 - Tela indicando o path e a biblioteca zip que acabou de chegar do GitHub	51
Figura 45 - Indicação de cliques para selecionar a porta USB.....	51
Figura 46 - Confirma a seleção da placa AI Thinker e clique na porta que aparece “(USB)”	52
Figura 47 - Jumper cinza coloca o microcontrolador no modo “programação”	52
Figura 48 - Nessa foto real, jumper cinza acionando o modo “programação”	53
Figura 49 - Botão reset no detalhe da seta vermelha. Pressione 1x após encaixar o jumper cinza	53
Figura 50 - Clique no ícone “seta para direita” para gravar o ESP32-CAM.....	54
Figura 51 - Código-fonte sendo compilado para depois ser gravado	55
Figura 52 - Código-fonte sendo gravado no ESP32-CAM.....	55
Figura 53 - Indicação de como se abre o MONITOR SERIAL do Arduino IDE	56
Figura 54 - Indicação de como alterar o <i>baud rate</i> do Monitor Serial de 9600 para 115200	57
Figura 55 - Tela do Monitor Serial indicando qual IP chegou. No destaque, use /cam-hi.jpg.....	57
Figura 56 - Imagens sendo capturadas pelo ESP32-CAM e visualizadas no navegador Google Chrome	58
Figura 57 - Local de armazenamento do best.pt dentro do Colab.....	61
Figura 58 - Precisamos instalar esses pacotes. Alguns serão inclusos em outros ...	62
Figura 59 - Teste final do reconhecedor usando uma segunda tela.....	63

LISTA DE TABELAS

Tabela 1 - Resultados do treinamento	38
--	----

ESP32

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Estrutura de pastas e subpastas no seu Google Drive	33
Código-fonte 2 - Importando o seu Google Drive ao seu Colab. Ambos deve pertencer à mesma conta Google	34
Código-fonte 3 - Fazendo clone do YOLO versão 5 do repositório do GitHub	34
Código-fonte 4 - Instalando os requerimentos de forma automática do YOLO5	34
Código-fonte 5 - Conteúdo do arquivo *.yaml. Atenção ao trecho em negrito.....	35
Código-fonte 6 - Indicação da organização do arquivo *.yaml	36
Código-fonte 7 - Comando para executar o treinamento do modelo com base nas imagens.....	36
Código-fonte 8 - Validação dos resultados.....	37
Código-fonte 9 - Indicação da organização da subpasta “test”	40
Código-fonte 10 - Código fonte do testado independente para o seu modelo	41
Código-fonte 11 - Resultado do teste do modelo	42
Código-fonte 12 - Código-fonte a ser gravado no ESP32-CAM	49
Código-fonte 13 - Código que roda o modelo YOLO em tempo real integrado ao ESP32-CAM	60
Código-fonte 14 - Instalando e atualizando o gerenciador Pip	61
Código-fonte 15 - Instalando o opencv no seu VS Code.....	62
Código-fonte 16 - Instalando o Torch	62
Código-fonte 17 - Instalando o Pandas, gerenciador de banco de dados.....	62
Código-fonte 18 - Instalando o Requests	62
Código-fonte 19 - Instalando o Pillow	62
Código-fonte 20 - Removendo versões incompatíveis do Numpy e do Scipy	62
Código-fonte 21 - Instalando uma versão específica	62

SUMÁRIO

1 ESP32: A JANELA PARA A VISÃO COMPUTACIONAL	7
1.1 ESP32-CAM	7
1.2 Lista de material	9
1.3 Limites do projeto	12
2 HANDS ON	16
2.1 Objetivos do projeto	16
2.2 Fluxo da instalação	16
2.2.1 Início.....	18
2.2.2 Coleta e organização de imagens.....	24
2.2.3 Anotação e rotulagem de dados.....	28
2.2.4 Treinamento e validação do modelo YOLO.....	33
2.2.5 Teste do modelo.....	40
2.2.6 Montagem do hardware.....	43
2.2.7 Integrando o ESP32-CAM com o modelo.....	59
3 APLICAÇÕES NO MUNDO REAL	64
CONCLUSÃO.....	66
REFERÊNCIAS.....	68

1 ESP32: A JANELA PARA A VISÃO COMPUTACIONAL

1.1 ESP32-CAM

A visão computacional tem se consolidado como uma das tecnologias mais impactantes da atualidade, permitindo que sistemas embarcados possam interpretar imagens e vídeos para tomar decisões automatizadas.

No contexto da Internet das Coisas (IoT), o uso de câmeras embarcadas tem se tornado uma solução viável para aplicações como monitoramento remoto, segurança, automação industrial e análise de padrões visuais.

Entre os dispositivos disponíveis no mercado para esse tipo de aplicação, o **ESP32-CAM** se destaca por oferecer uma alternativa de baixo custo, com conectividade Wi-Fi e Bluetooth, além de suporte para processamento de imagens em tempo real para pequenos projetos.

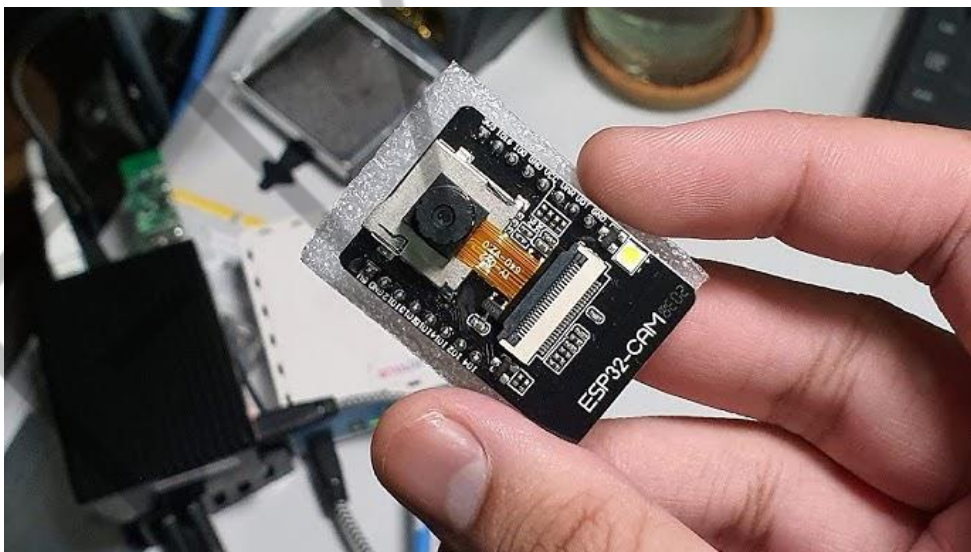


Figura 1 - ESP32 com uma pequena câmera (OV2640) de 2MP
Fonte: YouTube (2021)

O ESP32-CAM é um módulo baseado no microcontrolador ESP32, que integra uma câmera de baixa potência e um conjunto de recursos voltados para aplicações embarcadas.

ESP32: A Janela para a visão computacional

Esse módulo se diferencia por sua capacidade de capturar e processar imagens localmente, além de enviar dados para servidores remotos e operar de forma autônoma sem a necessidade de hardware adicional sofisticado.

Diferente de outras soluções populares, como o Raspberry Pi com câmeras dedicadas de 8MP ou Jetson Nano da NVIDIA com câmeras de 12MP que possuem maior capacidade de processamento gráfico, o ESP32-CAM se posiciona como uma opção enxuta, com sua câmera de 2MP para projetos que exigem eficiência energética e comunicação sem fio.



Figura 2 - Raspberry Pi 4, considerando o menor PC do mundo
Fonte: Uol (2024)

Enquanto o Raspberry Pi permite a execução de redes neurais complexas localmente e o Jetson Nano é otimizado para aplicações de Inteligência Artificial mais robustas, o ESP32-CAM se destaca em cenários onde a simplicidade e o consumo energético reduzido são fatores essenciais.

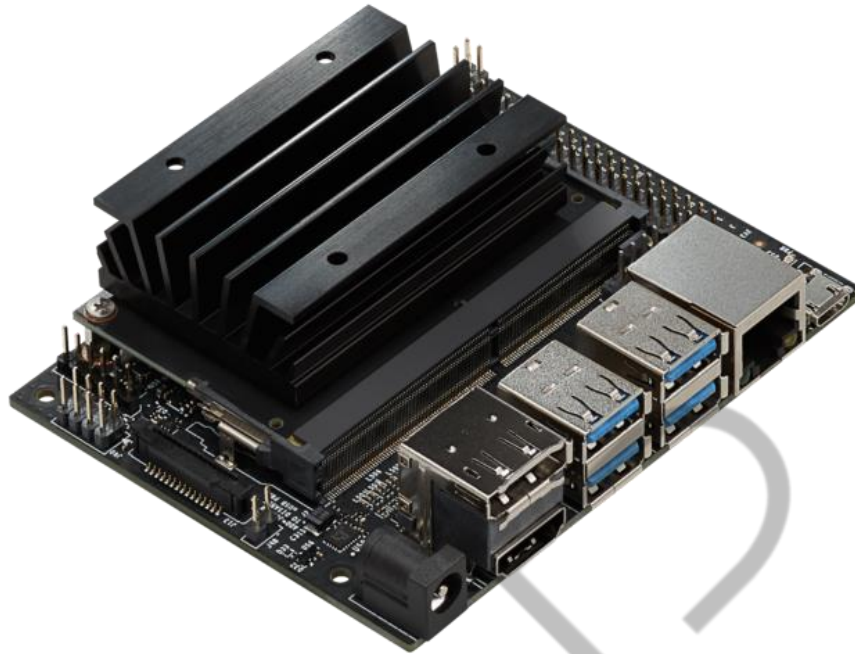


Figura 3 - Jetson Nano, da NVIDIA
Fonte: Nvidia Developer (2021)

Contudo, vale lembrar que, por enquanto, o ESP32-CAM não suporta modelos de Machine Learning em sua memória RAM. Ou seja, precisamos transmitir as imagens via Wi-Fi para um computador local, que por sua vez, executará uma Machine Learning em Python, de sua preferência.

1.2 Lista de material

Convidamos você e seu grupo para realizar esse Hands On de visão computacional e IoT. Vale lembrar que a compra não é obrigatória e nem está amarrada à distribuição de pontos.

Se desejar, poderá adquirir uma pequena lista de material, cujo investimento é relativamente baixo, cerca de R\$120,00 + frete para o seu CEP. Entretanto, mais cedo ou mais tarde, o avanço da trilha em busca do aprimoramento em IoT com IA dependerá da programação de placas reais, já que os simuladores gratuitos atuais não oferecem suporte ao ESP32-CAM para projetar um sistema de reconhecimento de objetos. É importante mencionar que você poderá participar das lives de tutoria para tirar qualquer dúvida relacionada ao projeto. A lista é a seguinte:

ESP32: A Janela para a visão computacional

- **01 ESP32-CAM com a minicâmera integrada OV2640, modelo AI Thinker.** Se no seu ESP32-CAM vier uma pequena antena, será aproveitada, porém, não é obrigatória.

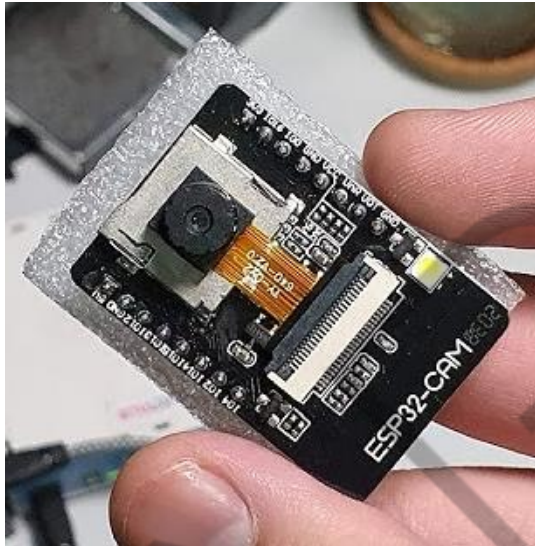


Figura 4 - ESP32-CAM com câmera OV2640 da AI Thinker
Fonte: YouTube (2021)

- **01 plaquinha da empresa FTDI**, que serve como interface de comunicação entre ESP32-CAM e o seu PC.

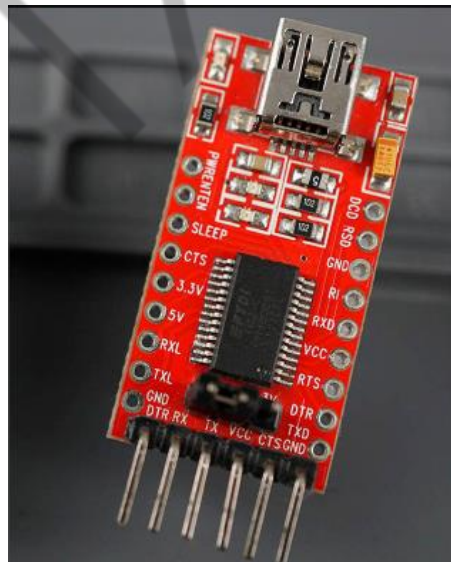


Figura 5 - Placa da empresa FTDI com conector micro USB
Fonte: Random Nerd Tutorials (2020)

- **05 jumpers fêmea-fêmea para Arduino de 10 cm de comprimento para ligar o adaptador USB e o ESP32-CAM.** Geralmente é vendido na Internet

em pacotinhos fechados de 40 jumpers coloridos. Os jumpers vêm pregados uns aos outros, mas você pode separá-los sem problema.



Figura 6 - Kit com 40 jumpers fêmea-fêmea coloridos
Fonte: Fatec (2023)

- **01 cabo USB compatível com o conector USB da plaquinha FTDI** que você optar, que será dentre as três opções: **mini USB**, ou **micro USB** ou **USB-C**. Na figura a seguir, mostramos os três tipos de cabo, onde você só providenciará aquele que se encaixa no FTDI que você comprou.

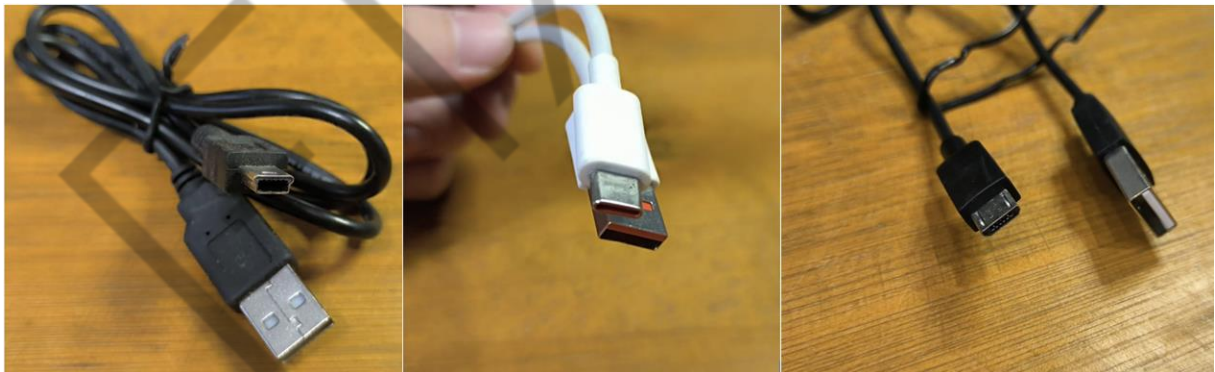


Figura 7 - Tipos de cabos USB. À esquerda, mini USB, ao centro, USB-C, à direita, micro USB
Fonte: Elaborado pelo autor (2025)

Dica: esse cabinho USB precisa ser de dados e não somente alimentação. Para testar se o cabo é de dados, conecte um celular ou outro dispositivo com armazenamento no seu PC usando esse cabo. Se o computador reagir, então é de dados. Se não, é somente de alimentação.

- **01 caixinha plástica saboneteira simples para você guardar e proteger o seu projeto depois de montado.** O tamanho da caixa é de

aproximadamente 7 cm de largura, 10 cm de comprimento e 4 cm de altura. Pode providenciar uma saboneteira um pouco maior sem problemas.



Figura 8 - Saboneteira plástica simples de qualquer cor
Fonte: Elaborado pelo autor (2025)

1.3 Limites do projeto

A arquitetura do ESP32-CAM é baseada no ESP32-S, um microcontrolador desenvolvido pela Espressif Systems, que incorpora um processador dual-core Xtensa LX6 rodando em até 240 MHz (um clock bem superior aos dos Arduinos tradicionais), acompanhado de uma RAM limitada (cerca de 520KB, por isso, ainda não suporta algoritmos de Machine Learning tradicionais).

Além da câmera integrada de 2MP, o módulo possui um slot para acomodar um micro cartão de memória SD de até 32GB (sistema FAT32). Esse cartão é para armazenar imagens ou vídeos que a aplicação do usuário pode solicitar, mas não para esse projeto.

Portanto, após a conexão da microcâmera e do microcartão (que são nativos e não podem ser removidos do ESP32-CAM), sobra 1 pino GPIO (General Purpose Input and Output) que permite a você, desenvolvedor, integrar algum sensor adicional, apenas do tipo digital, tornando-o versátil para projetos que combinam visão computacional e IoT. Por exemplo, você pode colocar um sensor de presença para detectar pessoas e assim tirar uma foto, ou um sensor de nível de líquidos, ou um sensor de vibração, sensor de luz, de chuva, de gás, de toque, de rotação, entre

diversos outros tipos de sensores. Vamos explorar mais sobre essas aplicações em seções futuras neste capítulo.

A comunicação via Wi-Fi permite a transmissão de imagens para servidores ou sistemas na nuvem, possibilitando aplicações como monitoramento remoto de ambientes, controle de acesso baseado em reconhecimento facial e análise de padrões em tempo real. A frequência de operação do Wi-Fi do ESP32-CAM é de 2,4GHz. Portanto, esse dispositivo não funciona em roteadores que trabalham apenas em 5GHz ou Wi-Fi roteados a partir de celulares da Apple.

A programação do ESP32-CAM é realizada principalmente por meio da Arduino IDE, e pode ser feita através da linguagem Python (que você já conhece), porém, aqui se chamará MicroPython ou Espressif IoT Development Framework (ESP-IDF) que é o framework oficial de desenvolvimento para programar e configurar dispositivos ESP32, incluindo o ESP32-CAM. Ele é mantido pela Espressif Systems e fornece ferramentas, bibliotecas e APIs para desenvolvimento avançado de aplicações de IoT.

Escolha o ambiente de programação que melhor lhe agrada, lembrando que essa escolha influencia o nível de controle e da complexidade desejada no projeto. Esse Hands On vai adotar o mais simples, que é o Arduino IDE.

Outra limitação é sobre como se comunicar com o ESP32-CAM. Como o módulo não possui uma porta USB nativa na plaquinha para realizar a programação, é necessário utilizar um **adaptador USB-Serial externo (fabricante FTDI que utiliza chips CP2102 ou CH340G)** e usar jumpers para conectá-los. A figura a seguir mostra as plaquinhas reais.

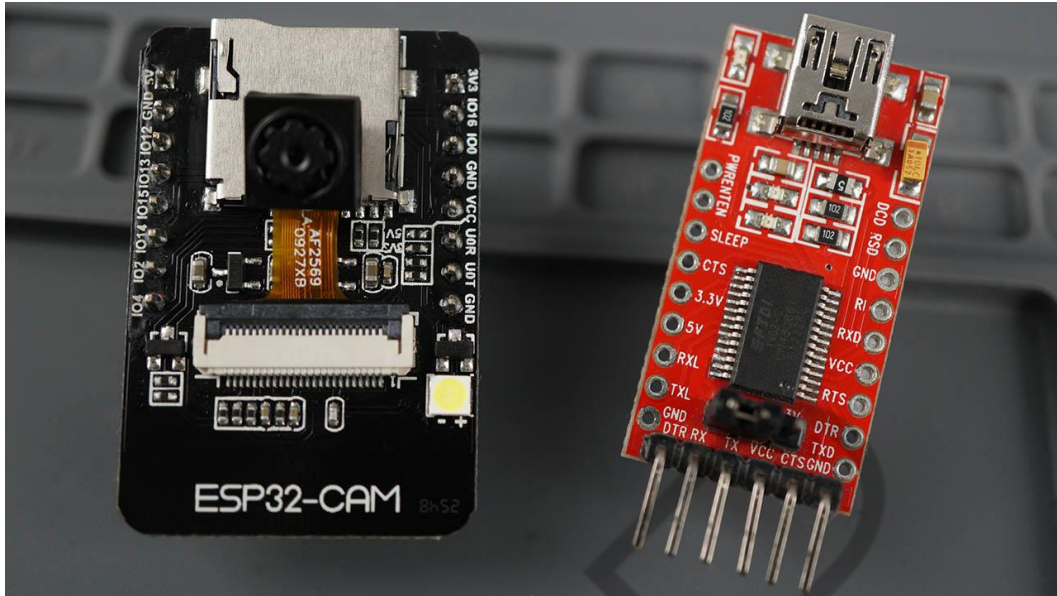


Figura 9 - ESP32 CAM à esquerda com a placa FTDI (vermelha) à direita
Fonte: Random Nerd Tutorials (2020)

O código para o ESP32-CAM segue a estrutura padrão da programação para ESP32 ou Arduino, isto é, utiliza as funções **setup()** e **loop()** como **obrigatórias**. É importante lembrar que no **setup()** são configurados o comportamento dos pinos, inicializada a câmera e estabelecida a conexão Wi-Fi, caso o projeto envolva transmissão de imagens para um servidor ou página web.

No **loop()**, são executadas funções repetitivas, como captura de imagens e envio para um destino remoto. Um dos exemplos mais comuns para este módulo é a criação de um **servidor web**, permitindo que o usuário visualize a imagem capturada pela câmera em tempo real por meio de um navegador. Veremos como fazer isso ainda nesse capítulo.

Para projetos mais avançados, como visão computacional com **YOLO** e **OpenCV**, a programação do ESP32-CAM pode envolver o envio de imagens para um servidor externo que realizará o processamento.

Como mencionado anteriormente, o ESP32-CAM tem recursos limitados para rodar Redes Neurais e Machine Learning diretamente. Nesse caso, a comunicação entre o ESP32-CAM e o servidor que hospedará e executará a detecção e classificação de objetos pode ser feita via **HTTP**, **MQTT** ou **WebSockets**, transmitindo os frames da câmera.

Outra alternativa para programação é o **MicroPython**, uma versão simplificada de Python otimizada para microcontroladores. Com ele, é possível desenvolver

aplicações de IoT utilizando menos código, embora seu suporte à câmera seja mais limitado do que na Arduino IDE. O ESP32-CAM vendido na Internet não está automaticamente pronto para interpretar o MicroPython.

Primeiramente, é necessário gravar outro firmware no ESP32-CAM usando a plaquinha FTDI e uma ferramenta de gravação chamada **esptool.py**. Em seguida, depois que estiver com o firmware que interprete o Python, você pode usar o Thonny para programar sua plaquinha usando Python. No GitHub e no Instructables.com existem repositórios que ensinam essa alternativa.

Já o **ESP-IDF**, o ambiente oficial da Espressif, oferece maior controle sobre o hardware e otimização de desempenho, sendo mais indicado para aplicações industriais e desenvolvimento avançado.

Em resumo, o **ESP32-CAM** pode ser programado de maneira acessível por meio da Arduino IDE para projetos simples ou utilizando frameworks mais robustos, como o ESP-IDF, para aplicações mais complexas. Para fins didáticos, vamos focar o nosso Hands On no Arduino IDE.

2 HANDS ON

2.1 Objetivos do projeto

Esse Hands On tem como objetivo principal o desenvolvimento de um sistema embarcado de detecção de objetos de sua escolha utilizando o modelo de redes neurais convolucionais YOLOv5 em conjunto com o microcontrolador ESP32-CAM.

Imagens capturadas em tempo real pela minicâmera do ESP32-CAM serão enviadas para um sistema instalado no seu notebook, que será capaz de identificar objetos específicos em tempo real.

Esse projeto explora habilidades práticas de aplicações de Visão Computacional, Aprendizado de Máquina e IoT, permitindo aplicações inovadoras na automação residencial, segurança, agronegócio, monitoramento industrial e outras áreas.

2.2 Fluxo da instalação

Nesta seção, temos um fluxograma com a visão geral do que será feito para concluir esse projeto, e um diagrama apresentando a arquitetura e as tecnologias que serão adotadas. Como mostra a figura “Fluxograma das etapas do projeto reconhecedor de objetos”, teremos sete etapas.

1. Início → instalação dos ambientes VS Code + Arduino IDE.
2. Coleta e Organização de Imagens → coleta de imagens do objeto ou da pessoa a ser reconhecido(a).
3. Anotações e Rotulações de Dados → processamento das imagens no <<https://www.makesense.ai/>> e, após isso, a exportação dos dados.
4. Treinamento e Validação do Modelo YOLO → preparação do Colab para treinar o modelo e na sequência, validar sua qualidade.
5. Teste do Modelo → ainda no Colab, fazer um teste de qualidade do modelo.
6. Montagem do Hardware → montagem do ESP32-CAM, gravação do código na plaquinha e teste da transmissão da imagem via Wi-Fi.

7. Integrando o ESP32-CAM com o Modelo → execução do reconhecedor de objetos ou pessoa no VS Code usando o ESP32-CAM.



Figura 10 - Fluxograma das etapas do projeto reconhecedor de objetos
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Na figura a seguir, você encontra a arquitetura do projeto e as tecnologias utilizadas, além de como elas estão organizadas em cada dispositivo.

No computador, vamos ter Arduino IDE, Visual Code, Python e suas bibliotecas relacionadas ao Opencv. Como hardware, teremos o ESP32-CAM da empresa IA Thinker com um adaptador USB externo da marca FTDI.

No centro, teremos um roteador comum, que pode ser residencial ou seu próprio celular, mas operando em 2,4GHz e não em 5GHz.

Na nuvem, teremos o uso do Google Drive, do app Make Sense AI e do Colab.

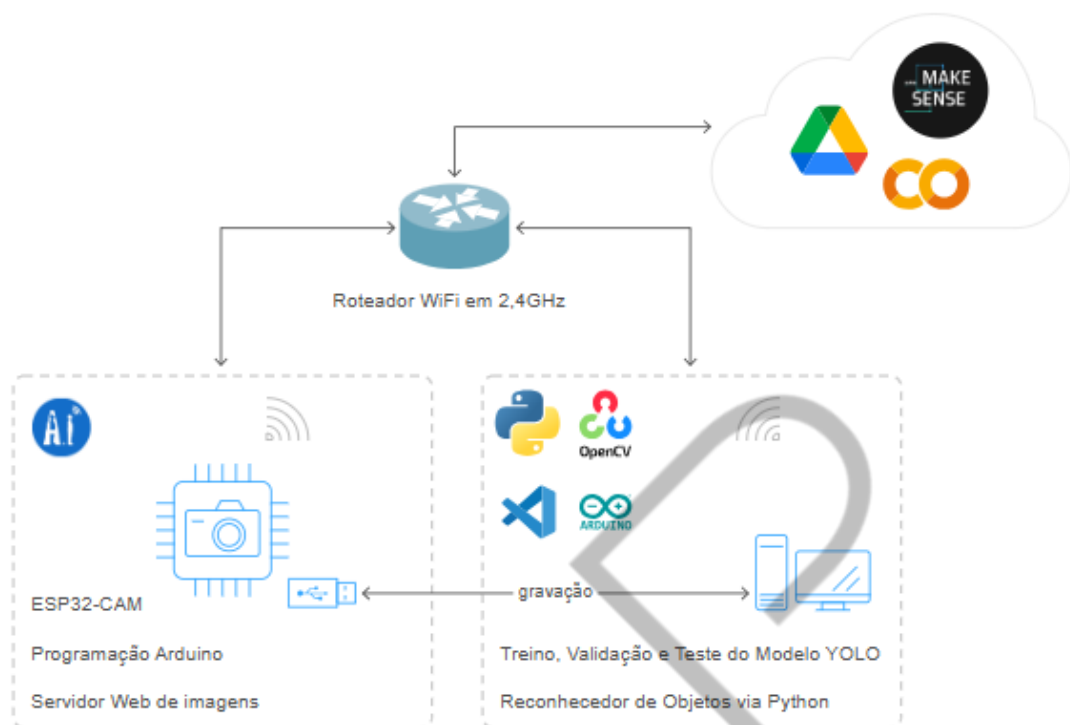


Figura 11 - Arquitetura do projeto e suas tecnologias
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Vamos dar início ao projeto prático de Visão Computacional.

2.2.1 Início

Nesta etapa, temos requisitos iniciais importantes a serem cumpridos, isto é, o que precisa ser instalado em seu computador. Vamos considerar o ambiente Windows numa arquitetura x86 de computador. Outros sistemas operacionais, como iMAC e Linux podem ser discutidos nas lives com o tutor e entre seus colegas de turma.

Nessa primeira etapa, vamos preparar e instalar o seu ambiente de desenvolvedor(a), começando pelo Visual Studio Code da Microsoft (VS). Se ainda não possui o VS, instale a versão mais recente, disponível em: <https://visualstudio.microsoft.com/pt-br/downloads/>.

Caso tenha o VS, mas não tenha um Python associado a ele, vá no menu esquerdo vertical, procure e clique no ícone Extensions (indicado pela seta da figura a seguir), depois digite “Python” no campo de pesquisa logo acima e instale-o. A versão do Python precisa ser a 3 ou superior.

ESP32: A Janela para a visão computacional

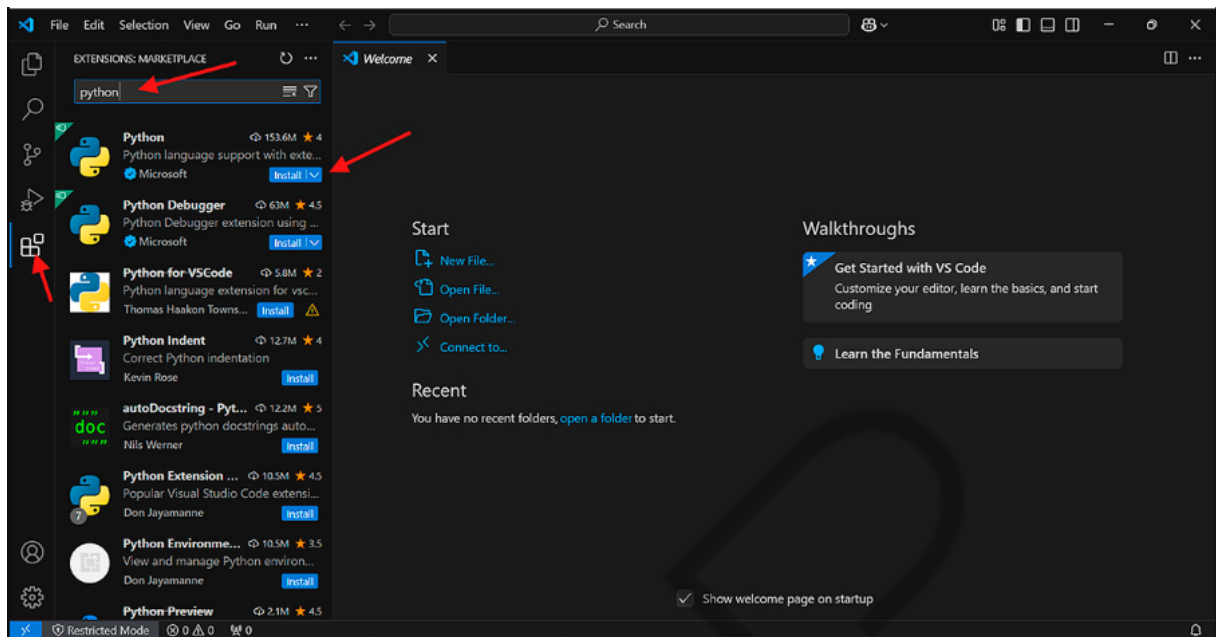


Figura 12 - Tela do Visual Studio indicando a instalação do Python
Fonte: Elaborado pelo autor (2025)

Após a instalação da extensão do Python, você pode conferir o que está instalado no seu PC expandindo o grupo “INSTALLED” no menu esquerdo vertical.

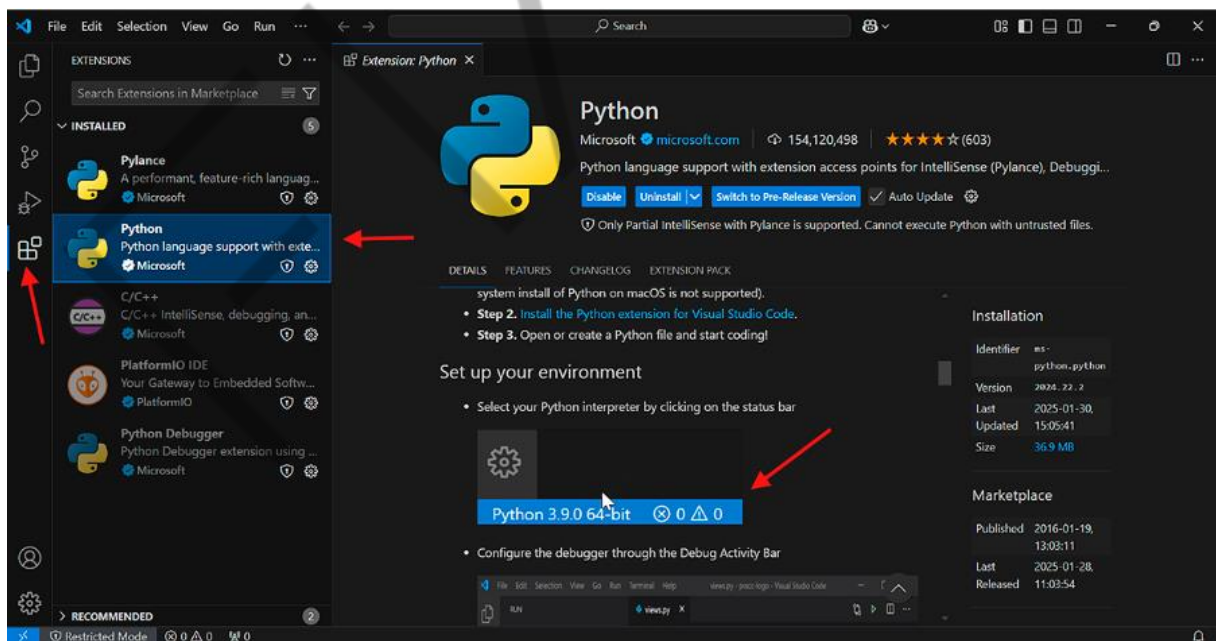


Figura 13 - Imagem do ambiente do VS indicando o que está instalado
Fonte: Elaborado pelo autor (2025)

Esse ambiente será utilizado mais para frente, quando for necessário rodar o detector de objetos. Por enquanto, vamos deixá-lo assim.

ESP32: A Janela para a visão computacional

Agora, vamos instalar o ambiente de programação de placas Arduino e ESP32, chamado Arduino IDE. Vá até o site <www.arduino.cc> e faça o download do arquivo de instalação mais recente do site indicado. Não precisa fazer doações em dinheiro e nem cadastro, apenas clique em “just download”. Após o download, o instalador possui algo próximo de 147MB de tamanho. Dê duplo clique no arquivo recém-baixado.

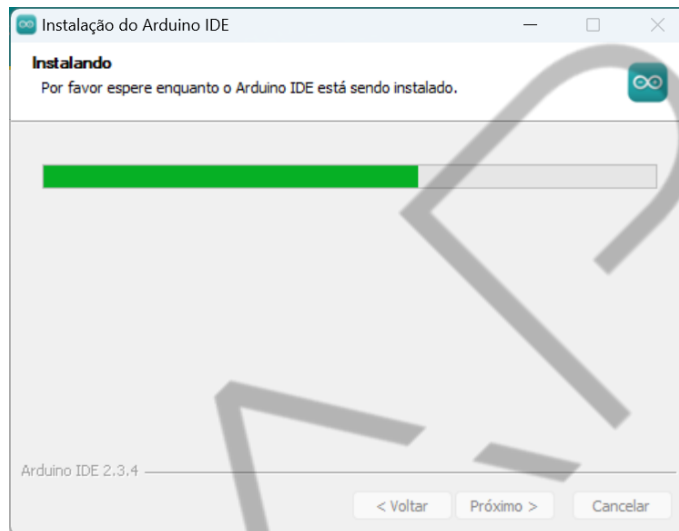


Figura 14 - Barra de progresso de instalação do Arduino IDE
Fonte: Elaborado pelo autor (2025)

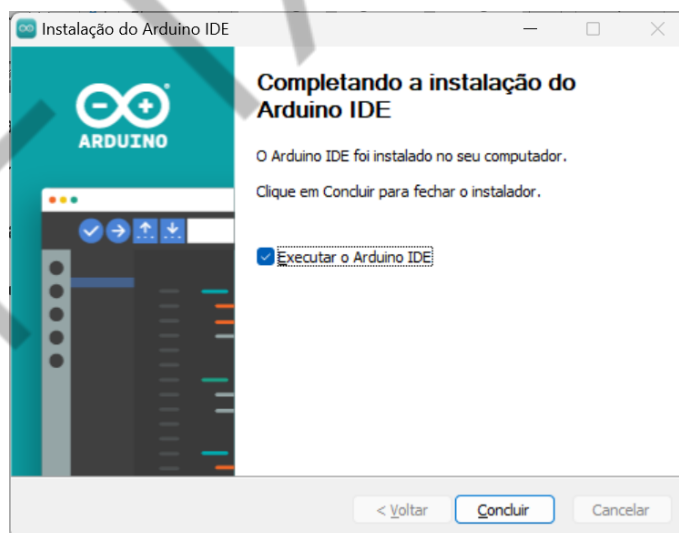


Figura 15 - Tela de conclusão da instalação
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Qualquer demais telas de pop-up de segurança do Windows que aparecer em seu PC, vai dando permissão de instalação ou acesso de administrador. A tela a seguir é só um exemplo, mas podem aparecer várias outras.

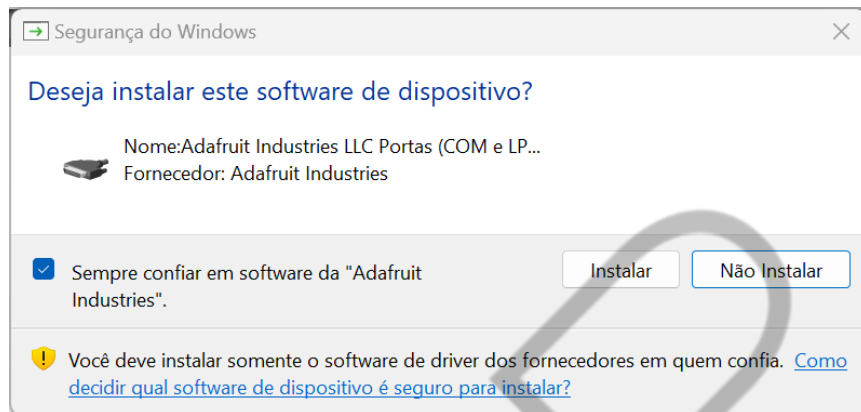


Figura 16 - Exemplo de tela de segurança do Windows solicitando permissão
Fonte: Elaborado pelo autor (2025)

Concluída a instalação do Arduino IDE, vamos instalar a placa ESP32-CAM, que não vem instalada nativamente no IDE. Para isso, clique nos ícones ou campos indicados pelas setas vermelhas na figura a seguir. No campo indicado, digite “esp32” sem o “cam”. Não se preocupe com a versão 3.1.1 que aparece na figura, pois já pode haver versões mais recentes dependendo do momento que estiver lendo esse texto. O importante é sempre pegar a mais recente.

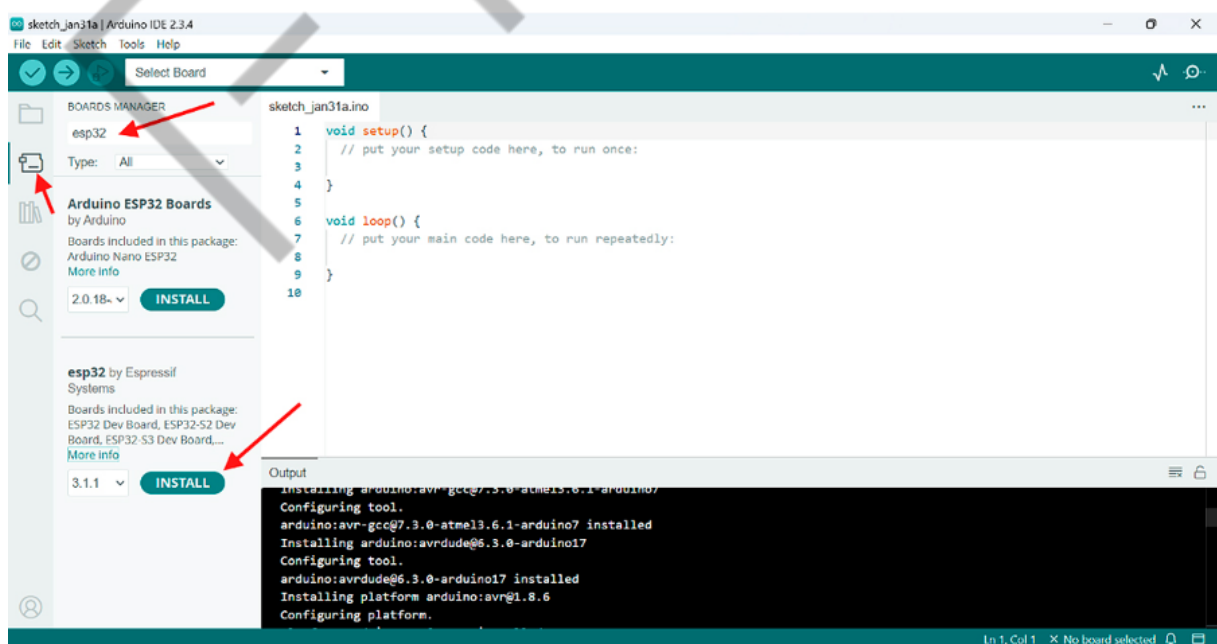


Figura 17 - Cliques e preenchimentos que devem ser feitos para encontrar a família ESP32
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Você observará uma tela igual a ilustrada na figura “Download da família ESP32 sendo realizado”, aguarde o download finalizar e instalar.

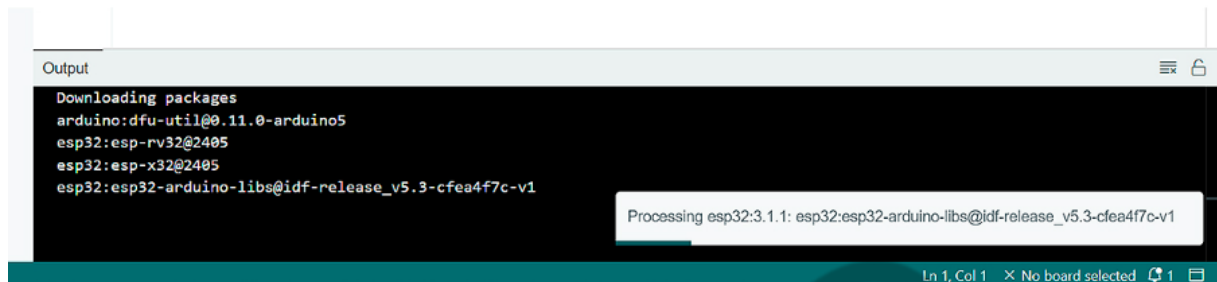


Figura 18 - Download da família ESP32 sendo realizado
Fonte: Elaborado pelo autor (2025)

A instalação será automática. A dica para saber se a instalação foi concluída com sucesso é observar se a família de placas ESP32 está instalada em sua IDE, conforme a seta vermelha da figura a seguir.

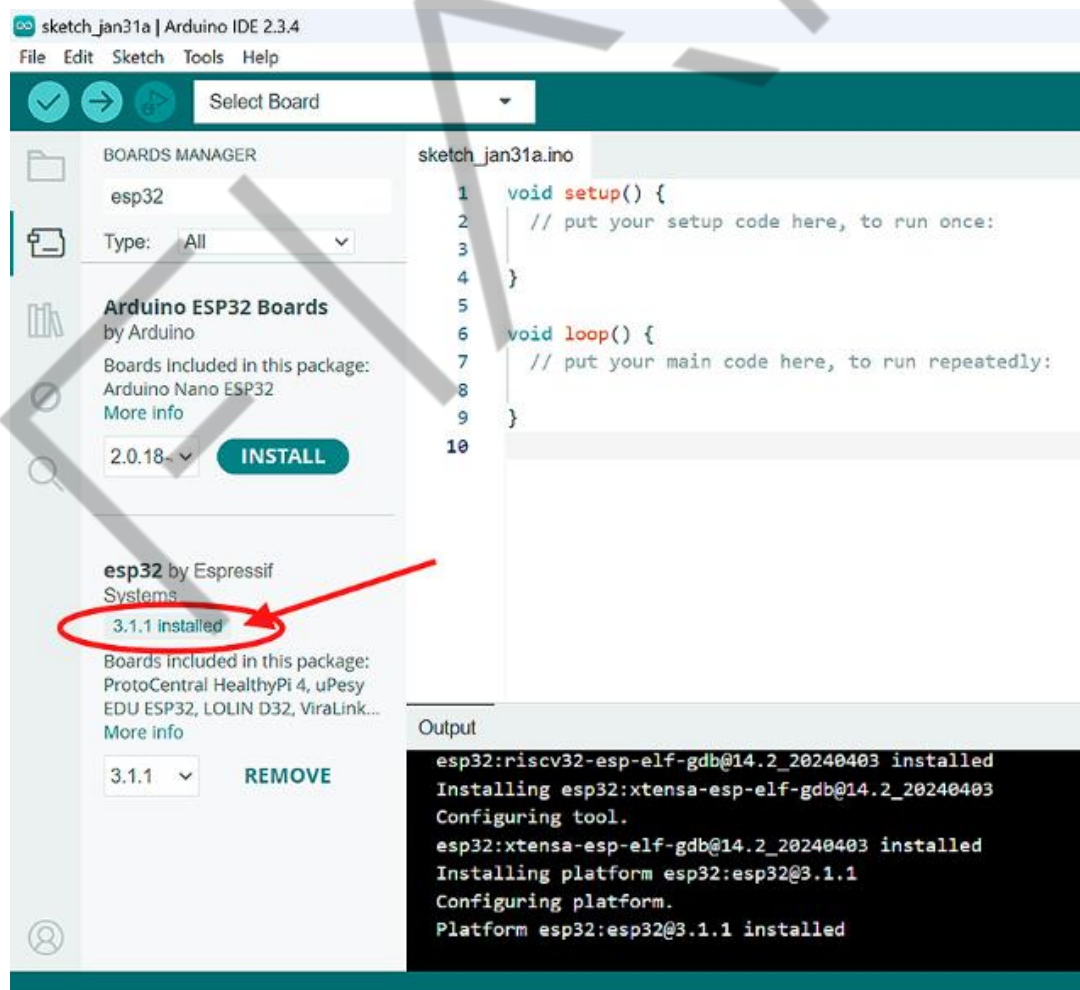


Figura 19 - Dica para saber se a família de placas ESP32 foi instalada corretamente
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Outra dica para saber se a instalação ocorreu dentro dos conformes, é seguindo as setas vermelhas da figura a seguir. Clique em “Select Board”, depois “Select other board and port...”.

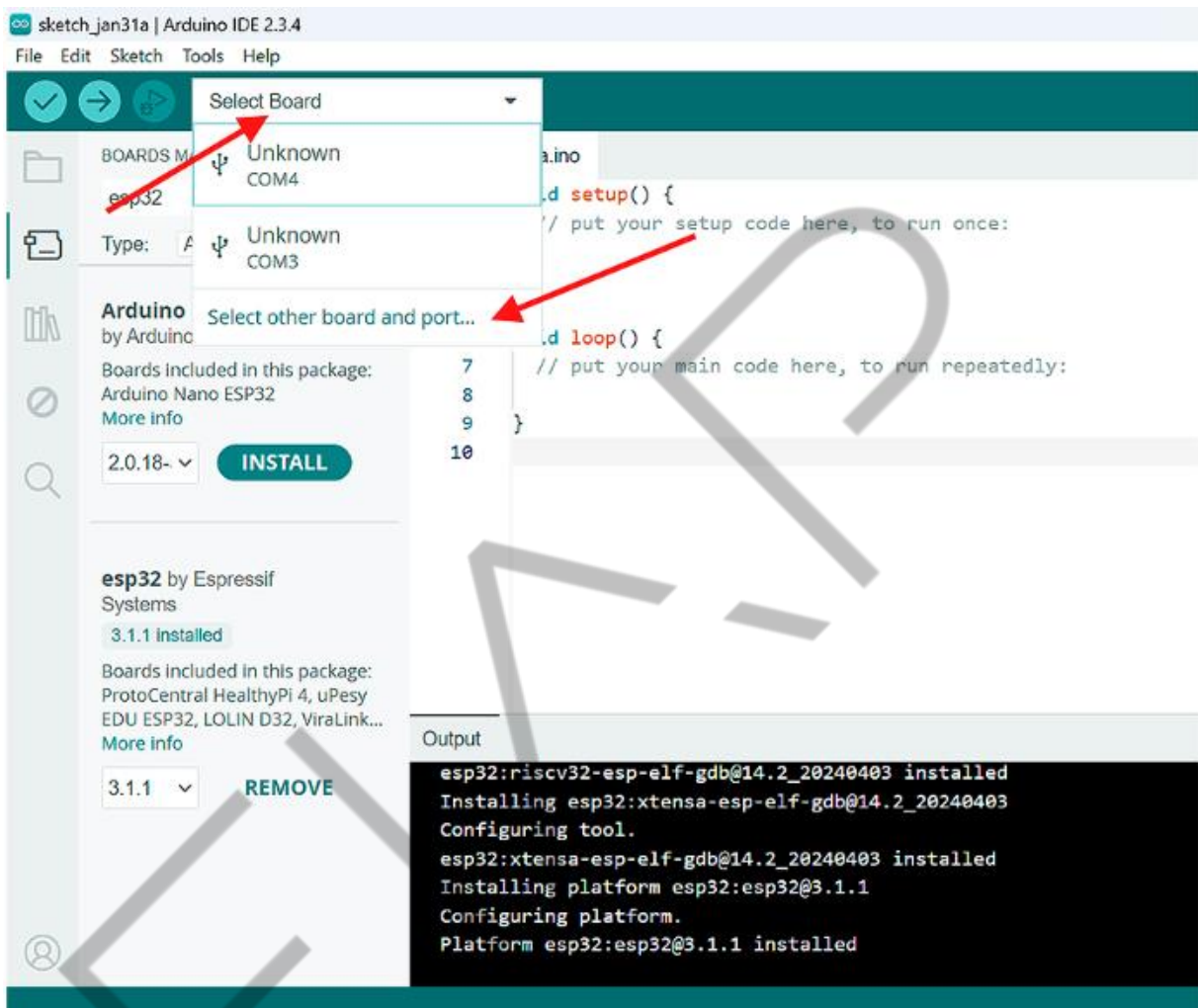


Figura 20 - Conferindo se a placa ESP32-CAM foi instalada corretamente
Fonte: Elaborado pelo autor (2025)

Na sequência, uma outra tela aparecerá, onde você deverá digitar “ai thinker” e, se tudo correu bem, ela vai aparecer logo abaixo como mostra a figura “Encontrando a plaquinha ESP32-CAM da AI Thinker”. Já pode clicar na plaquinha e deixá-la selecionada e confirme no OK.

ESP32: A Janela para a visão computacional

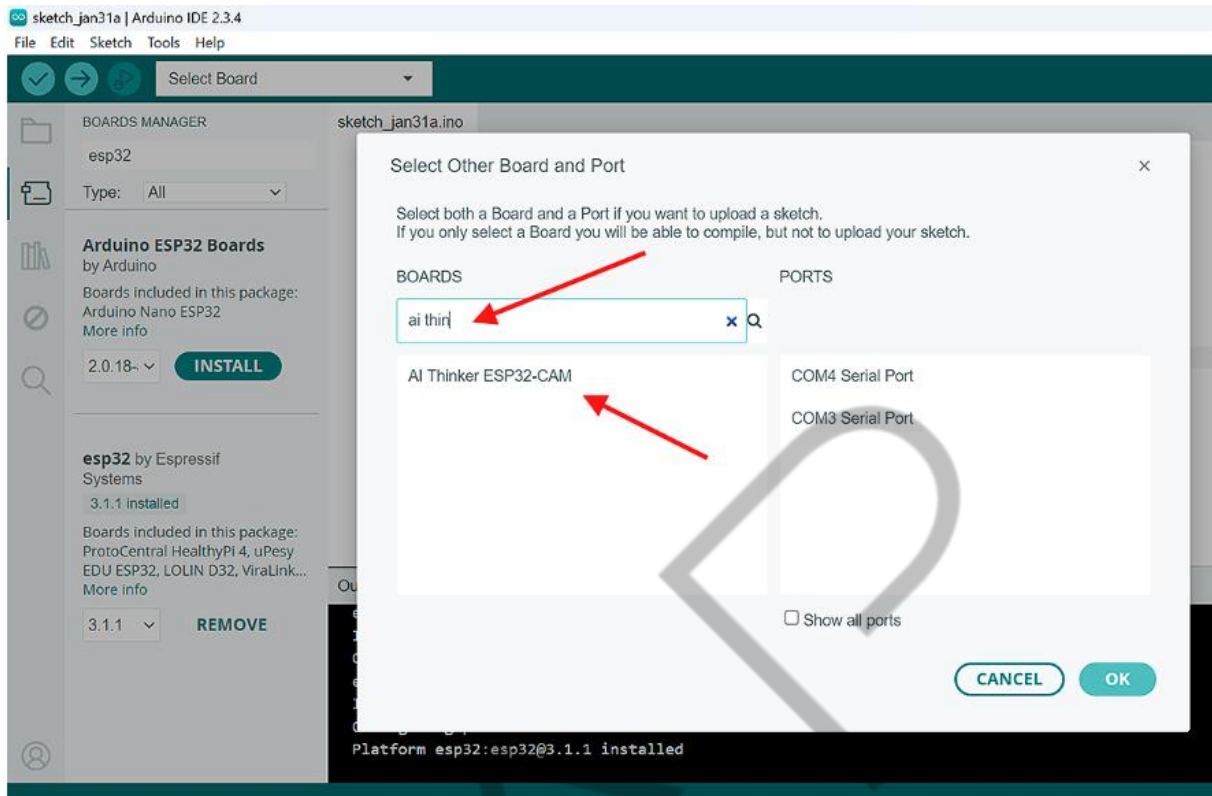


Figura 21 - Encontrando a plaquinha ESP32-CAM da AI Thinker
Fonte: Elaborado pelo autor (2025)

Pronto! Se você observa a AI Thinker ESP32-CAM conforme a figura anterior, seu ambiente Arduino IDE está concluído e tudo certo para ser utilizado para programar a plaquinha ESP32-CAM.

2.2.2 Coleta e organização de imagens

A detecção de objetos baseada em **Deep Learning** requer um conjunto de imagens para treinar o modelo e um outro conjunto para testar o modelo. Essa captura de imagens pode ser feita pela sua câmera de celular ou buscando imagens na Internet.

Caso você queira reconhecer um objeto específico em seu ambiente ou uma pessoa, terá que usar a câmera do seu celular. Vale mencionar que o seu celular não precisa estar na qualidade máxima de resolução, mas, uma alternativa caso não queira usar seu celular para coletar imagens e deseja reconhecer um objeto simples, como um telhado de casa, pode buscar vários telhados na Internet ou utilizar um banco de imagens à sua escolha.

O importante é providenciar, no mínimo, 40 imagens de um objeto de sua escolha, de diversos ângulos. De frente, de trás, de lado, de cima e de baixo. Tire fotos em um ambiente com boa iluminação. Nomeie as figuras como **fig1.jpg**, **fig2.jpg**... **fig40.jpg**. Pode usar a extensão *.png também. Essas 40 imagens serão utilizadas para a etapa de treinamento.

Providencie outras quatro imagens do mesmo objeto (fig41, fig42, fig43 e fig44), também de diversos ângulos. Essas quatro imagens serão utilizadas para validar o seu sistema de reconhecimento. Não podem ser as mesmas imagens de treinamento. Caso utilize as mesmas imagens do treinamento para a etapa de validação, o desempenho poderá ser comprometido.

Providencie mais outras quatro imagens para testar o seu sistema (fig45, fig46, fig47 e fig48.jpg).

Validar é diferente de testar:

- **Validar** o modelo significa verificar se ele generaliza bem ao tentar reconhecer um objeto que ele nunca viu antes. Essa validação é feita automaticamente logo após o treinamento e usando as próprias ferramentas do Opencv.
- **Testar** o modelo significa verificar o resultado da acurácia e se está generalizado e se reconhece imagens usando uma ferramenta de terceiros e independente da biblioteca Opencv.

Então, quando você utiliza as imagens do treinamento também no teste ou na validação, a chance é de ter apenas impactos negativos, tais como:

- **Superestimação da Precisão (Overfitting):** se o modelo já viu as imagens de teste durante o treinamento, ele pode memorizar padrões específicos dessas imagens em vez de aprender regras gerais para detecção de objetos. Isso leva a uma **precisão artificialmente alta** nos testes, mas um desempenho ruim em imagens novas.
- O modelo pode funcionar bem **apenas** com imagens do conjunto de treinamento, mas falhar ao encontrar objetos em **novos contextos** (com

iluminação, ângulos ou cenários diferentes). Em aplicações reais, ele pode **não detectar corretamente objetos em imagens inéditas**.

- A métrica de **mAP (mean Average Precision)** e outras estatísticas podem **não refletir a verdadeira qualidade do modelo**, pois a avaliação está sendo feita com dados já conhecidos. Isso dificulta a identificação de problemas e impede a melhoria do modelo.
- O modelo pode aprender **características irrelevantes das imagens de treinamento**, como **texturas, ruídos ou marcas específicas**, em vez de padrões genéricos dos objetos. Isso pode fazer com que a IA **cometa erros graves** quando exposta a novas imagens.
- **Problemas na Implantação:** um modelo treinado e testado com os mesmos dados pode parecer funcional na fase de desenvolvimento, mas falha no mundo real. Isso pode impactar sistemas de segurança, reconhecimento facial, inspeção industrial e outras aplicações críticas.
- **Como evitar esse problema?** Separando as imagens em **treinamento (80%), validação (10%) e teste (10%)**. Nesse Hands On, teremos 40 imagens para treinamento, +4 imagens inéditas para validação e +4 imagens inéditas para testes, resultando no total de 48 imagens do mesmo objeto, porém de ângulos e iluminações diferentes. Mas você pode aumentar essa proporção a depender da acurácia que deseja atingir. Nesse exemplo, nossa acurácia deverá estar entre 60 e 70%, a depender da iluminação do local, tanto no treinamento quanto na aplicação.
- Aplicar **transformações nas imagens** (rotação, escala, brilho, inversão) para garantir que o modelo aprenda a reconhecer o objeto sob diferentes condições ajuda e muito. Esse processo chama-se Data Augmentation.

Para armazenar as imagens, vamos utilizar o seu Google Drive pessoal, porque lá na frente será necessário usar o Colab, visto que ele facilmente se conecta ao seu Google Drive.

Crie a seguinte estrutura de pasta (images) e subpastas (train e val) para você armazenar as imagens:

ESP32: A Janela para a visão computacional

- images/train → nessa pasta, guarde as 40 imagens inéditas iniciais.
- images/val → nessa, guarde outras 4 imagens inéditas.

A figura “Ambiente do Google Drive com as pastas e subpastas” apresenta uma ideia de como ficaria no Google Drive.

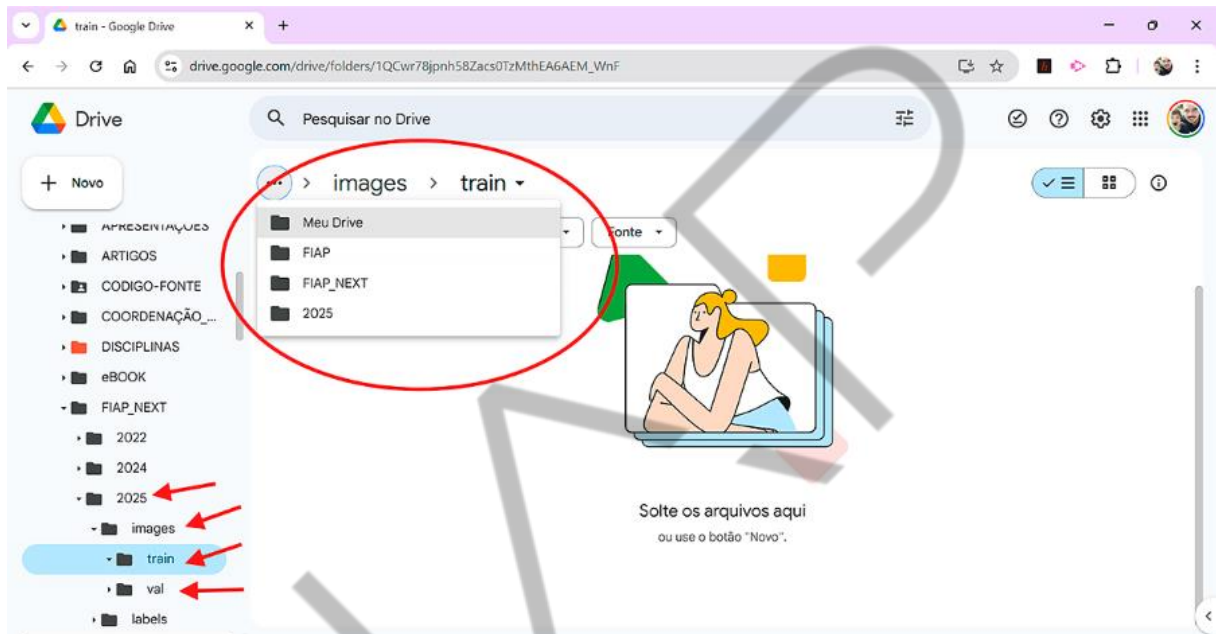


Figura 22 - Ambiente do Google Drive com as pastas e subpastas
Fonte: Elaborado pelo autor (2025)

Agora, povoe suas subpastas, preenchendo com as 40 imagens de treino como **fig1.jpg**, **fig2.jpg**, **fig3...** e **fig40.jpg**. Pode usar a extensão *.png também, e utilize as outras **quatro imagens** do mesmo objeto (**fig41**, **fig42**, **fig43** e **fig44**) para a subpasta “val”, conforme mostra a figura a seguir.

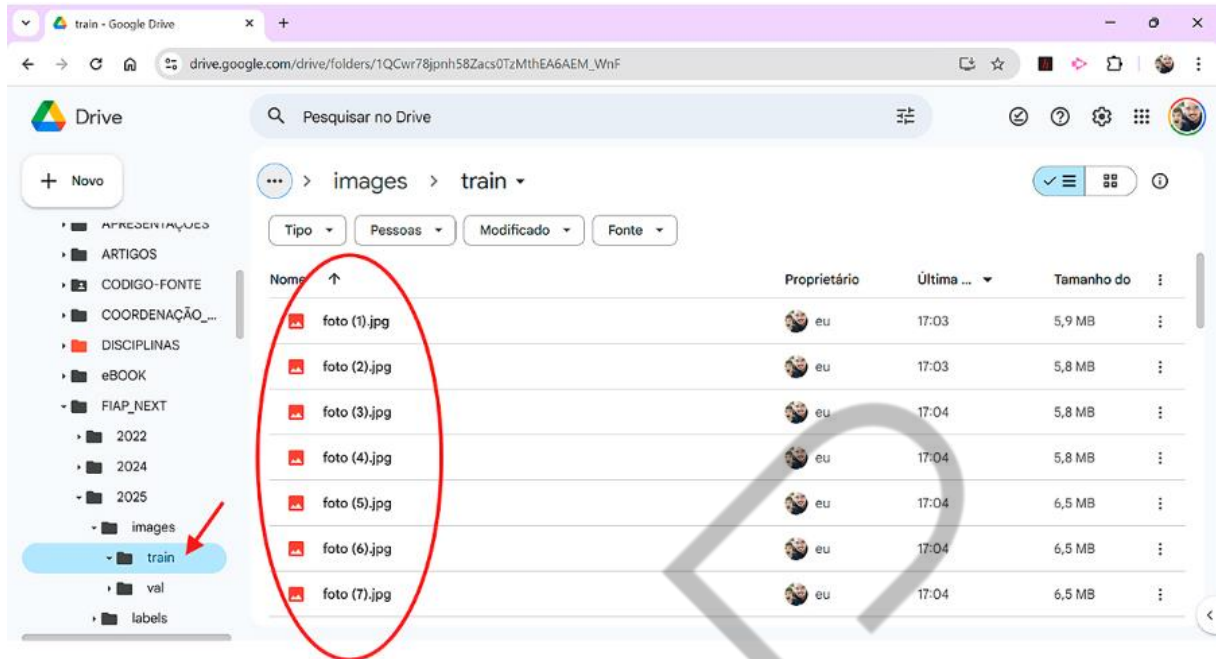


Figura 23 - Povoando a subpasta de treino com X imagens
Fonte: Elaborado pelo autor (2025)

Depois de tudo pronto, faça o download das subpastas “train” e “val” para o seu PC, e descompacte as pastas e deixe-as reservadas para logo mais. Isso será temporário e, no final, você poderá apagá-las.

Essa etapa temporária é para você fazer a transferência das imagens para o site makesense.ia que será na etapa seguinte.

2.2.3 Anotação e rotulagem de dados

O modelo YOLOv5 precisa de imagens **anotadas**, ou seja, com caixas delimitadoras (**bounding boxes**) que indicam onde está o objeto na imagem. Isso é particularmente interessante porque limita a quantidade de pixels que precisa ser analisada dentro de uma imagem, tornando o modelo mais magro.

Portanto, isso facilitaria na coleta de imagens, onde você poderia ter um arquivo de imagem com dois objetos sendo usados para o treinamento. Isso só faz sentido se você estiver utilizando um banco de imagens onde existem várias imagens de telhado no mesmo arquivo, por exemplo, ou há um telhado e uma pessoa no mesmo arquivo e você gostaria de aproveitar o trabalho e já resolver para os dois ao mesmo tempo usando o mesmo arquivo, como mostra a figura a seguir.



Figura 24 - Imagem com 2 objetos, boneco e um telhado
Fonte: Elaborado pelo autor (2025)

O programa que vamos usar para fazer essa caixa delimitadora é um aplicativo online que não precisa ser instalado. Ele está disponível nesse link: <https://www.makesense.ai/>.

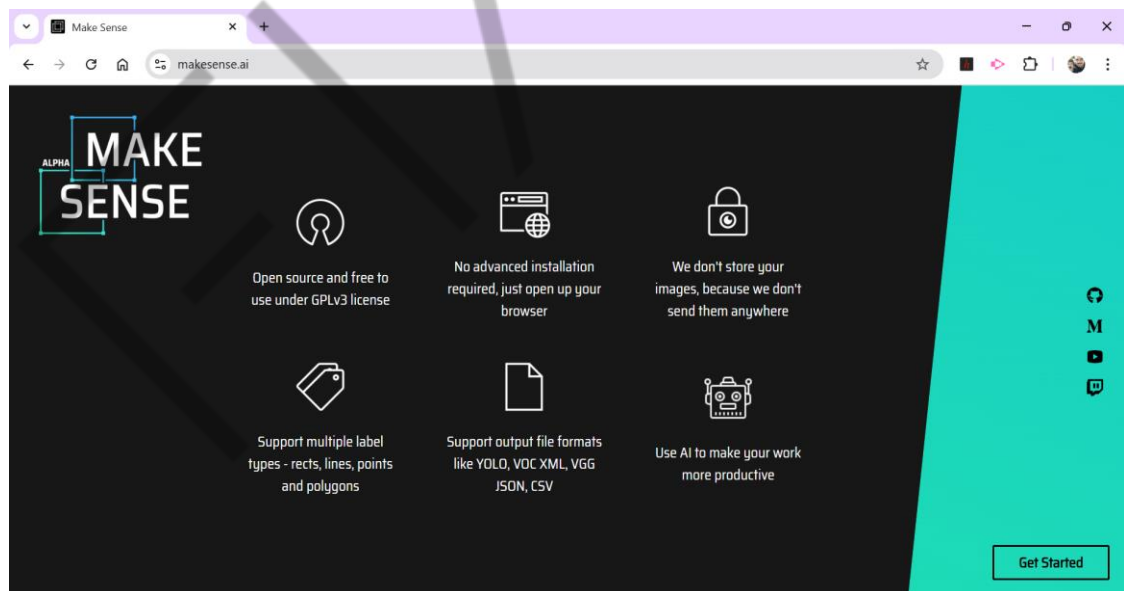


Figura 25 - Site do Make Sense. Pode haver alguma atualização do layout sem aviso prévio
Fonte: makesense.ia (2025)

Clique no botão “Get Started” e, em seguida, clique na caixa “Drop images or click here to Select them”.

ESP32: A Janela para a visão computacional

Aponte para a pasta de treino, e selecione todas as imagens digitando “Ctrl + a” e clique em Abrir.

Na sequência, clique em “OBJECT DETECTION”. Agora, você vai criar o label, que é o nome do objeto que você pretende reconhecer. Nesse exemplo, vamos reconhecer uma pessoa e o telhado.

Então, clique no **sinal de +** (mais), adicione o primeiro label e clique novamente no sinal de +, e adicione quantos labels quiser. Para sair da tela, vá em “Start project”.

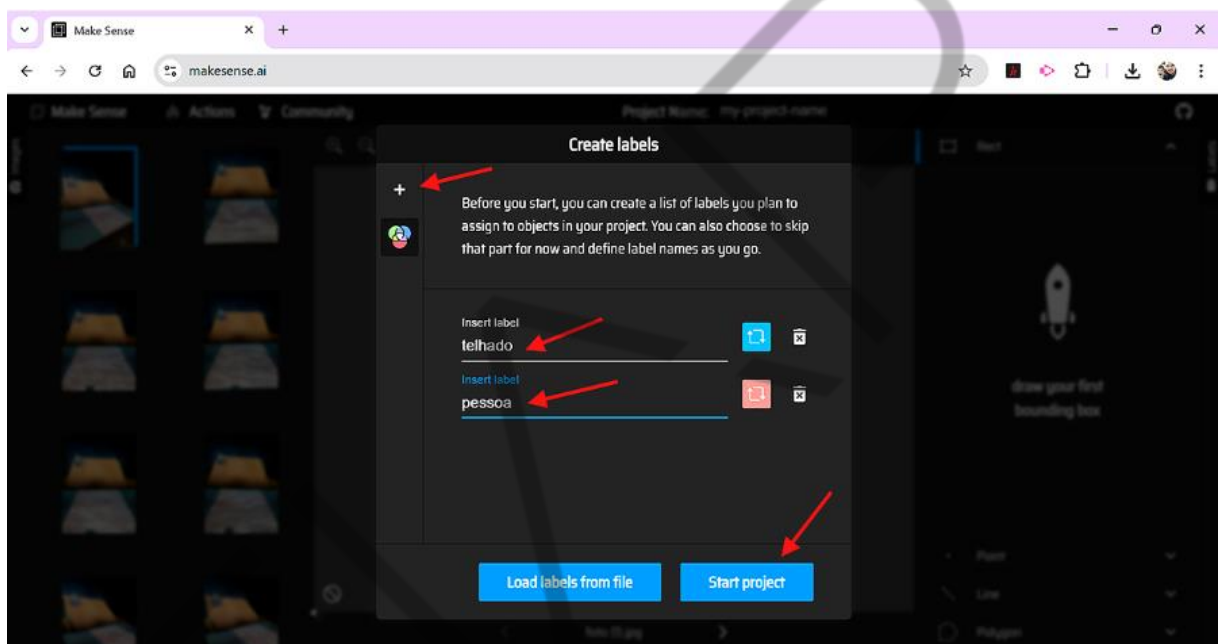


Figura 26 - Definindo os rótulos das imagens
Fonte: Elaborado pelo autor (2025)

Faça um retângulo em volta de cada objeto do cenário (**não pode ser polígono, porque é incompatível com o YOLO**) e associe à respectiva label no canto direito vertical, como mostra a figura “Criando as áreas limitantes dos objetos e associando ao seu label”. Em outras palavras, faça uma demarcação no boneco e classifique-o como “pessoa”, faça uma demarcação no telhado e classifique-o como “telhado”. Repita esse processo para cada uma das 40 imagens do seu banco de imagens de treino. Lógico que você é livre para fazer com o cenário e a imagem que quiser.

ESP32: A Janela para a visão computacional

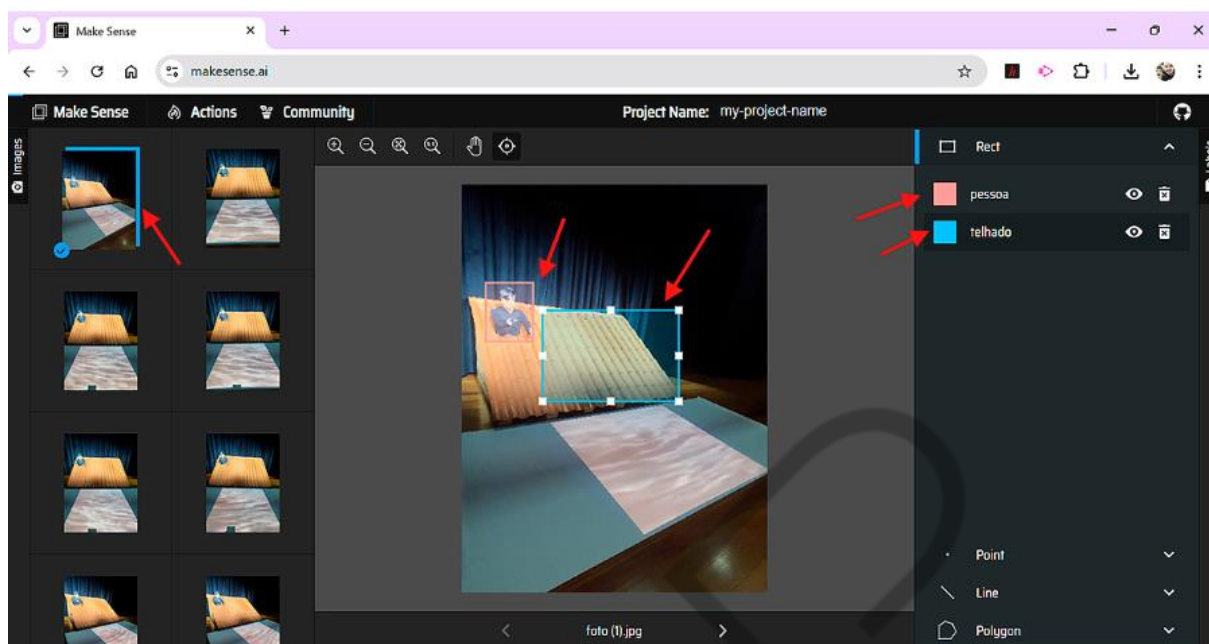


Figura 27 - Criando as áreas limitantes dos objetos e associando ao seu label
Fonte: Elaborado pelo autor (2025)

Quando terminar, clique no menu “Actions” para exportar os dados anotados, isto é, os arquivos que contém as coordenadas XY delimitando onde está cada imagem a ser processada no futuro.

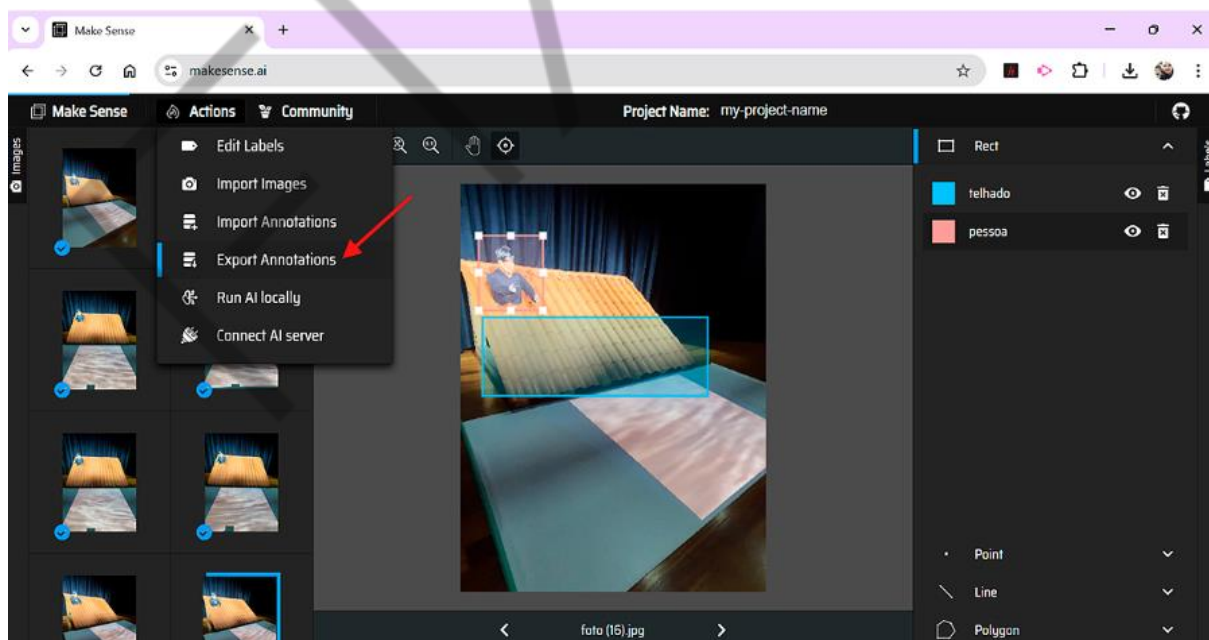


Figura 28 - Exportando as coordenadas XY delimitadoras das imagens
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Clique em “Export annotations” e indique o padrão YOLO e salve no mesmo lugar que você salvou provisoriamente as imagens de treino. Veja a figura a seguir para entender melhor.

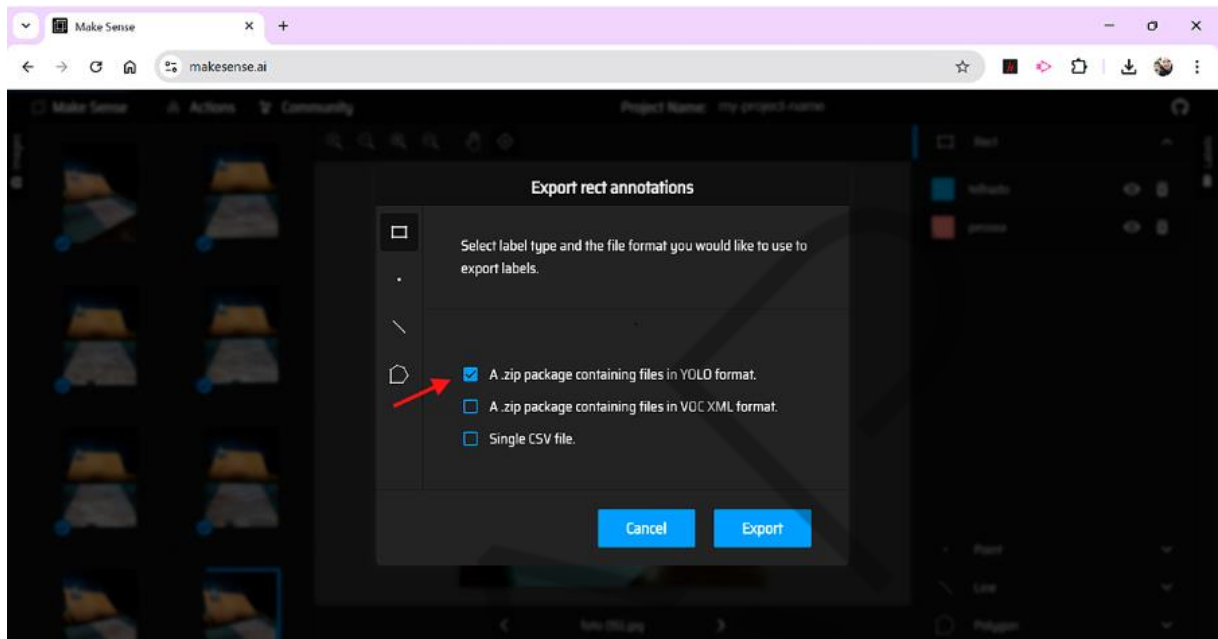


Figura 29 - Exportando no padrão do YOLO
Fonte: Elaborado pelo autor (2025)

Agora, retorne ao início dessa subseção e faça o mesmo processo de delimitação das imagens da pasta teste que você salvou provisoriamente no seu computador, baixado do Google Drive.

No final dessa etapa, você precisa ter em seu computador dois arquivos *.ZIP, sendo um correspondente aos labels das imagens de treino e outro correspondente aos labels das imagens de validação. Note que os labels possuem os mesmos nomes das fotos originais, mas agora, eles possuem a extensão TXT. Não podemos diferenciar os nomes dos arquivos labels em relação aos nomes das fotos originais. A não ser que você altere o nome da foto e do correspondente label, aí sim!

Faça o upload dessas duas subpastas de labels do treino e da validação para o seu Google Drive, como mostra a figura a seguir. Mas atenção, ao fazer o upload, primeiramente descompacte os arquivos ZIP do makesense.io e, em seguida, suba descompactado para o seu Google Drive, se não, vai subir apenas um único arquivo de dentro do ZIP.

O resultado das pastas e subpastas ficará assim:

```
Home/2025/  
  /images  
    /train  
    /val  
  labels  
    /train  
    /val
```

Código-fonte 1 - Estrutura de pastas e subpastas no seu Google Drive
Fonte: Elaborado pelo autor (2025)

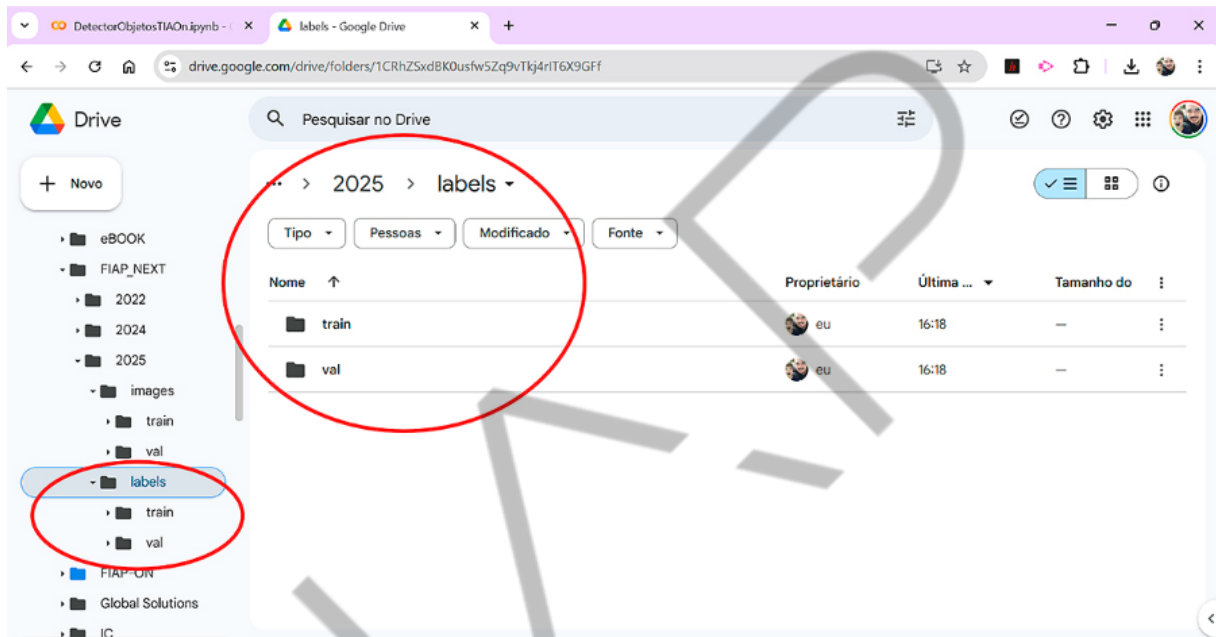


Figura 30 - Criação da pasta labels e subpastas treino e teste no Google Drive
Fonte: Elaborado pelo autor (2025)

Feito isso, vamos partir para o treinamento do YOLO!

2.2.4 Treinamento e validação do modelo YOLO

Com as imagens anotadas, isto é, com os seus respectivos labels, vamos iniciar o treinamento do modelo YOLOv5 utilizando o Colab e todas as quatro subpastas do seu Google Drive.

A primeira coisa que você precisa fazer é abrir um notebook Colab novinho e criar uma célula de código para cada bloco a seguir.

ESP32: A Janela para a visão computacional

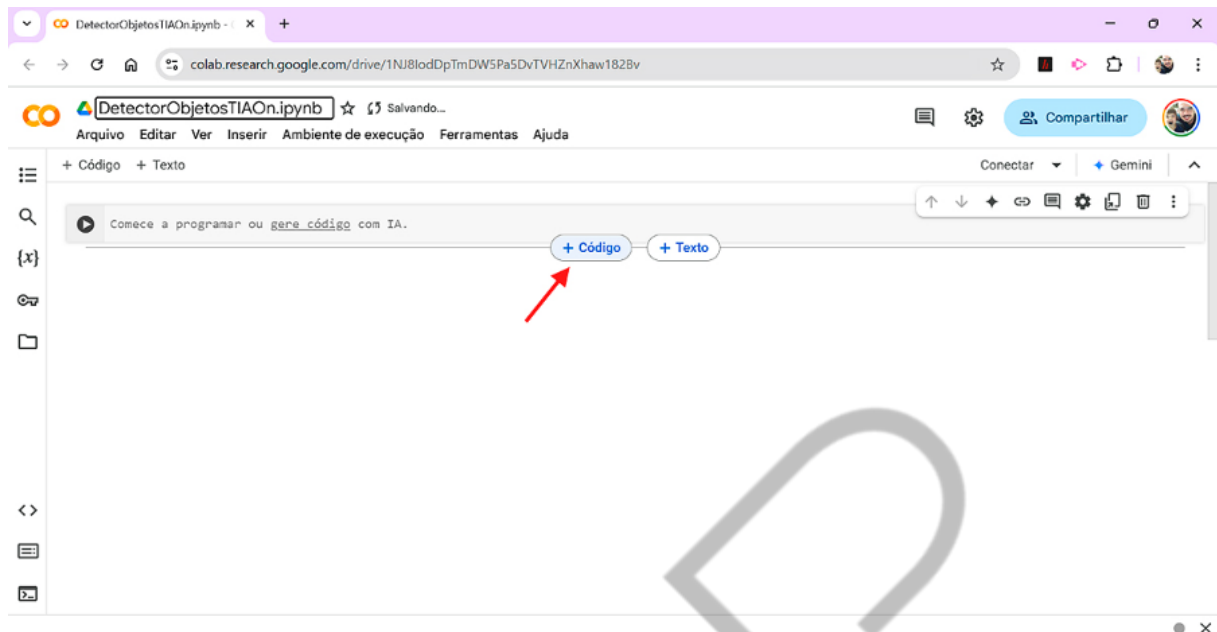


Figura 31 - Criando um Colab novo e células de código
Fonte: Elaborado pelo autor (2025)

Execute cada célula de forma separada para facilitar a identificação de falhas. O Colab irá lhe pedir permissão para acessar o seu driver logo ao executar a primeira célula abaixo, e sim, autorize associando com a sua conta Google que estão as subpastas de imagens.

```
from google.colab import drive
drive.mount('/content/drive')
```

Código-fonte 2 - Importando o seu Google Drive ao seu Colab. Ambos deve pertencer à mesma conta Google

Fonte: Elaborado pelo autor (2025)

```
! git clone https://github.com/ultralytics/yolov5.git
```

Código-fonte 3 - Fazendo clone do YOLO versão 5 do repositório do GitHub

Fonte: Elaborado pelo autor (2025)

```
! pip install -r yolov5/requirements.txt
```

Código-fonte 4 - Instalando os requerimentos de forma automática do YOLO5

Fonte: Elaborado pelo autor (2025)

Agora, instancie um computador mais poderoso que o tradicional. Instancie uma GPU T4, clicando em Alterar Tipo de Ambiente de Execução, como mostra a figura a seguir.

Dica: se ficar alguns minutos sem mexer no seu Colab, ele vai se desconectar. Então, execute as células desde o início se caso o Colab começar a dar falhas no código.

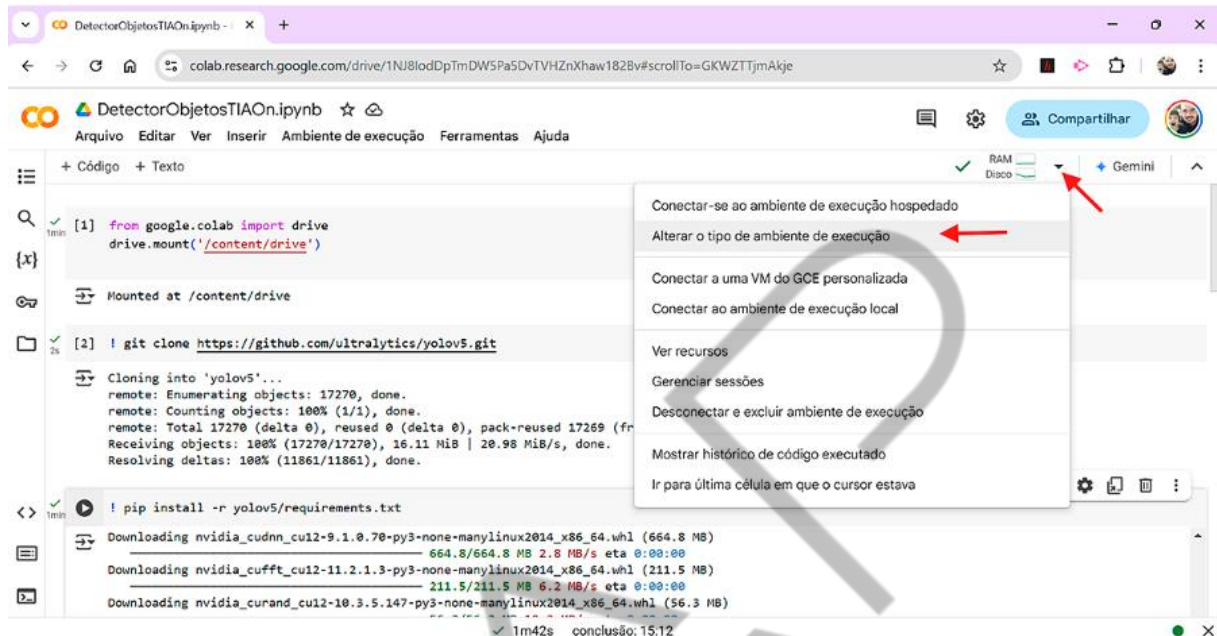


Figura 32 - Instanciando um computador mais poderoso no Colab, isto é, GPU T4
Fonte: Elaborado pelo autor (2025)

Voltando às subpastas do seu Google Drive, precisamos criar um arquivo chamado **telhado.yaml**. Note que a extensão do arquivo é *.YAML que é uma extensão específica, cuja função é indicar o path do seu dataset de imagens usado para a etapa de treinamento do modelo. Para criar esse arquivo, use o Bloco de Notas do seu PC, insira o código a seguir e altere a extensão de .TXT para .YAML e salve-o como **telhado.yaml**

```
train: /content/drive/MyDrive/FIAP/FIAP_NEXT/2025/images/train/
val: /content/drive/MyDrive/FIAP/FIAP_NEXT/2025/images/val/

names: # Defina aqui as labels do seu dataset
  0: "Telhado"
  1: "Pessoa"
  # ...
```

Código-fonte 5 - Conteúdo do arquivo *.yaml. Atenção ao trecho em negrito
Fonte: Elaborado pelo autor (2025)

Importante: esse trecho em negrito do path **/content/drive/MyDrive/** deve ser igual para você também porque é o mesmo para todos os usuários do Colab.

ESP32: A Janela para a visão computacional

Esse arquivo especial deve ficar na raiz das subpastas “train” e “val” aí do seu Google Drive. Exemplo:

```
Home/2025
  /images
    /train
    /val
  /labels
    /train
    /val
  /telhado.yaml
```

Código-fonte 6 - Indicação da organização do arquivo *.yaml

Fonte: Elaborado pelo autor (2025)

Após ter criado o arquivo **telhado.yaml**, arraste-o para a raiz do seu Google Drive conforme a figura a seguir.

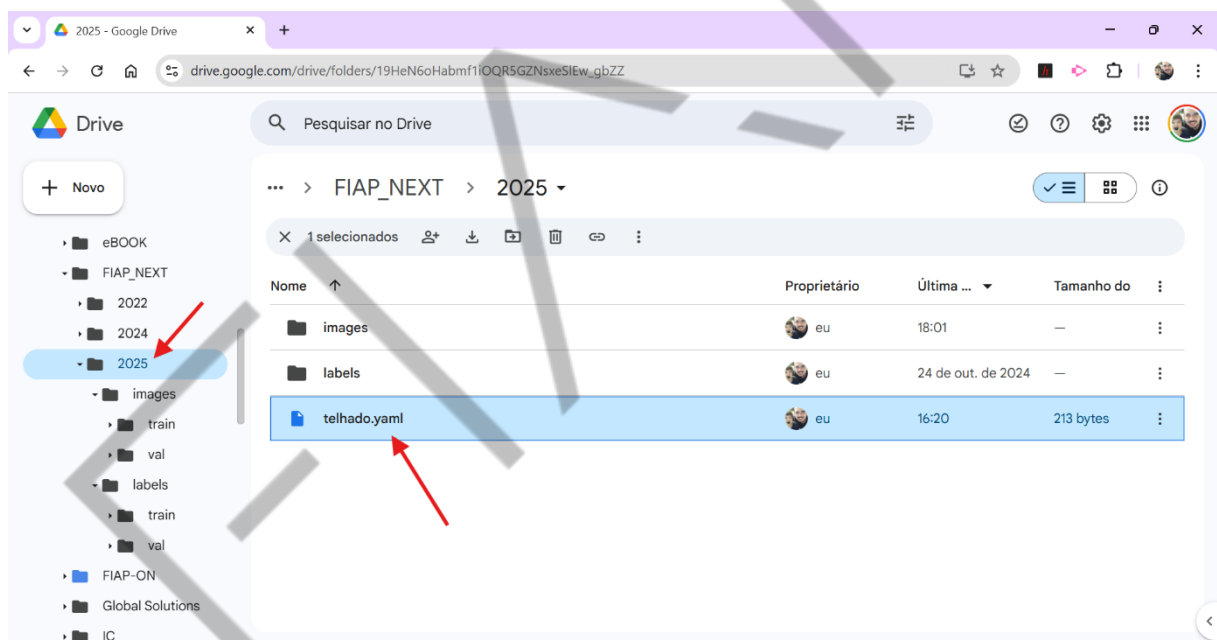


Figura 33 - Local do arquivo telhado.yaml no seu Google Drive. Mesmo nível das subpastas

Fonte: Elaborado pelo autor (2025)

No próximo código, você fará o treinamento do seu modelo. Você pode mexer na variável --epochs mudando o número na frente dele. Exemplo, você pode subir o **40** para **60** e analisar os resultados de acurácia do seu modelo.

```
!python yolov5/train.py --data telhado.yaml --weights yolov5s.pt
--img 640 --epochs 40
```

Código-fonte 7 - Comando para executar o treinamento do modelo com base nas imagens

Fonte: Elaborado pelo autor (2025)

Importante: é provável que apareça um prompt no Colab informando que há depreciação de bibliotecas. Por enquanto, não se preocupe com isso, e informe a opção “3” no prompt.

Tenha paciência nessa etapa porque demora mesmo e você pode acompanhar as épocas no prompt de resultados do seu Colab.

Qual é o resultado dessa etapa? No final da execução desse último código-fonte, é apresentada a seguinte mensagem:

```
Validating yolov5/runs/train/exp/weights/best.pt...
Fusing layers...
Model summary: 157 layers, 7015519 parameters, 0 gradients, 15.8 GFLOPs
```

	Class	Images	Instances	P	R
mAP50	mAP50-95: 100% 1/1	[00:00<00:00,	4.90it/s]		
	all	4	8	0.667	0.75
0.751	0.528				
	Telhado	4	4	0.939	0.75
0.754	0.602				
	Pessoa	4	4	0.395	0.75
0.748	0.455				

```
Results saved to yolov5/runs/train/exp
```

Código-fonte 8 - Validação dos resultados
Fonte: Elaborado pelo autor (2025)

Significa que o processo de treinamento foi concluído e o sistema está agora avaliando o desempenho dos melhores pesos do modelo, que foram salvos no caminho especificado (**yolov5/runs/train/exp/weights/best.pt**).

O YOLOv5 utiliza várias camadas em sua arquitetura. Essa etapa combina algumas dessas camadas para otimização e eficiência durante a inferência.

Resumo do modelo: 157 camadas, 7.015.519 parâmetros, 0 gradientes, 15,8 GFLOPs. Essa linha fornece um resumo da arquitetura do modelo treinado:

- **157 camadas:** número total de camadas na rede neural.
- **7.015.519 parâmetros:** quantidade total de pesos ajustáveis aprendidos pelo modelo durante o treinamento.
- **0 gradientes:** indica que o modelo não está sendo treinado no momento (os gradientes são usados para atualizar os pesos durante o treinamento). Em outras palavras, quando um modelo de rede neural como o YOLOv5

está sendo **treinado**, ele passa por um processo chamado **backpropagation** (retropropagação). Esse processo ajusta os **pesos** da rede neural para melhorar a precisão da detecção de objetos. Para fazer esse ajuste, o modelo calcula **gradientes** — que são basicamente as “correções” que precisam ser feitas nos pesos do modelo. Nesse projeto, não tivemos “correções” e dificilmente vamos ter, porque o algoritmo não está configurado para isso.

- **15,8 GFLOPs:** medida da complexidade computacional do modelo (bilhões de operações de ponto flutuante por segundo).

Classificação dos resultados:

A próxima seção apresenta as métricas de avaliação do desempenho do modelo nos dados de validação:

Classe	Imagens	Instâncias	Precisão (P)	Recall (R)	mAP50	mAP50-95
Todas	4	8	0,667	0,75	0,751	0,528
Telhado	4	4	0,939	0,75	0,754	0,602
Pessoa	4	4	0,395	0,75	0,748	0,455

Tabela 1 - Resultados do treinamento
Fonte: Elaborado pelo autor (2025)

- **Classe:** os nomes dos objetos que o modelo foi treinado para detectar ('todas', 'Telhado', 'Pessoa').
- **Imagens:** número de imagens usadas na avaliação. Nesse exemplo, foram quatro.
- **Instâncias:** quantidade total de objetos presentes nas imagens de validação para cada classe, que foram quatro para telhado e quatro para pessoa, totalizando oito.
- **P (Precisão):** das previsões feitas pelo modelo para uma determinada classe, qual proporção estava correta? O telhado se saiu melhor com 93,9% de acerto e 39,5% para a pessoa.

- **R (Recall):** das instâncias reais de uma determinada classe no conjunto de validação, quantas foram corretamente detectadas pelo modelo? 75% foram detectadas corretamente.
- **mAP50 (média de precisão média em IoU 0.5):** métrica comum em detecção de objetos, indicando a precisão média considerando um limiar de sobreposição (**IoU**) de 0.5 entre a caixa delimitadora prevista e a caixa real. Para esse exemplo, aproximadamente 75% de precisão para todos.
- **mAP50-95:** média da precisão em diferentes valores de IoU variando de 0.5 a 0.95, fornecendo uma visão mais ampla do desempenho do modelo. O telhado ganhou com 60% e a pessoa com 45,5% de acurácia.

Os resultados da avaliação e outros artefatos do treinamento foram salvos no diretório especificado (**yolov5/runs/train/exp**). Nesse diretório, você pode encontrar arquivos como os melhores pesos do modelo, logs de treinamento e o mais, visualizações da validação. Por exemplo:

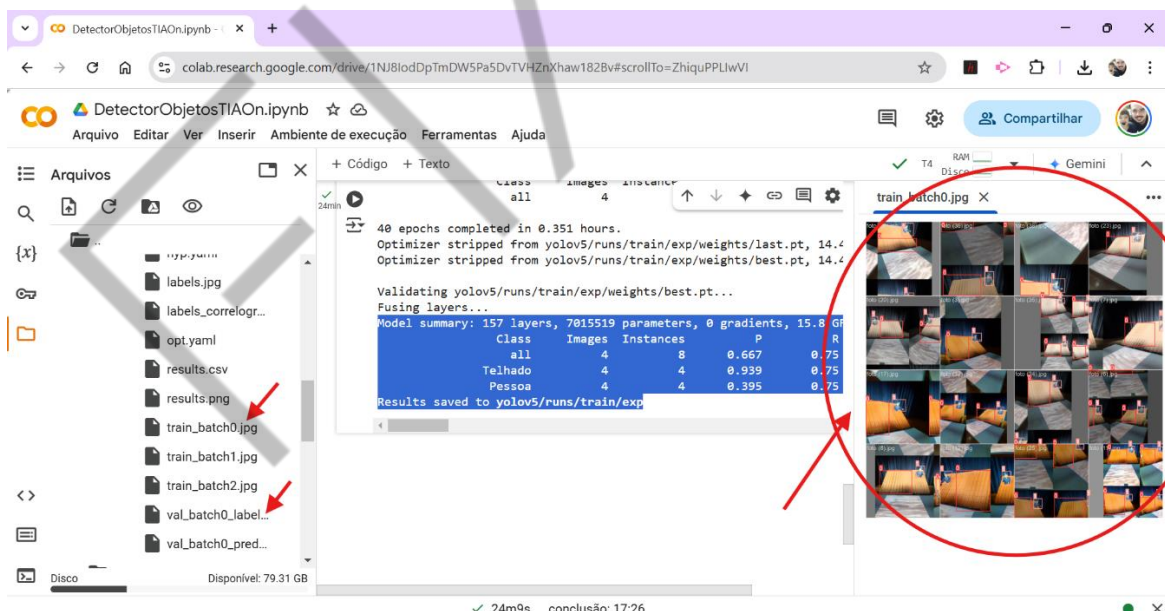


Figura 34 - Caixas delimitadoras reconhecendo cada objeto do cenário
Fonte: Elaborado pelo autor (2025)

Para melhorar a acurácia do seu modelo, precisa de mais e mais imagens de treino, com rotações e ângulos diferentes.

2.2.5 Teste do modelo

Após o treinamento, é necessário avaliar se o modelo consegue **detectar corretamente** os objetos nas imagens de teste usando um “software de terceiros”.

Primeiramente, vamos criar uma última subpasta na raiz de “image” chamada “test” e, em seguida, iremos povoar essa subpasta com as quatro últimas imagens inéditas do seu objeto.

Seu Google Drive ficará assim:

```
Home/2025
  /images
    /train
    /val
  /labels
    /train
    /val
  /telhado.yaml
  /test
    fig45.jpg
    fig46.jpg
    fig47.jpg
    fig48.jpg
```

Código-fonte 9 - Indicação da organização da subpasta “test”
Fonte: Elaborado pelo autor (2025)

Agora, copie o próximo código-fonte, cole na última célula do Colab e atualize as duas linhas que apontam o argumento **--source**. Você deve colocar o path da subpasta “test” de acordo com o seu Google Drive nessas linhas. O código a seguir destaca em negrito onde está o argumento **--source**.

```
import os
import subprocess

def get_latest_train_run_folder():
    subfolders = [f.path for f in
os.scandir('yolov5/runs/train') if f.is_dir()]
    latest_folder = max(subfolders, key=os.path.getctime,
default=None)
    return latest_folder

latest_run = get_latest_train_run_folder()
result = subprocess.run(f'python yolov5/detect.py --weights
{latest_run}/weights/best.pt --img 640 --source
drive/MyDrive/FIAP/FIAP_NEXT/2025/test/ --data
yolov5/data/telhado.yaml', shell=True, capture_output=True,
text=True)
if latest_run:
    # COMANDO
    result = subprocess.run(f'python yolov5/detect.py --weights
{latest_run}/weights/best.pt --img 640 --source
drive/MyDrive/FIAP/FIAP_NEXT/2025/test/ --data
yolov5/data/telhado.yaml', shell=True, capture_output=True,
text=True)
    print(result.stdout)
    print(result.stderr)
else:
    print("Não foi possível encontrar a pasta de treinamento
mais recente.")
```

Código-fonte 10 - Código fonte do testado independente para o seu modelo
Fonte: Elaborado pelo autor (2025)

Após executar a célula no Colab, esse será o resultado:

```
detect:      weights=['yolov5/runs/train/exp/weights/best.pt'],
source=drive/MyDrive/FIAP/FIAP_NEXT/2025/test/,
data=yolov5/data/telhado.yaml,      imgsz=[640,      640],
conf_thres=0.25,      iou_thres=0.45,      max_det=1000,      device=,
view_img=False, save_txt=False, save_format=0, save_csv=False,
save_conf=False, save_crop=False, nosave=False, classes=None,
agnostic_nms=False,      augment=False,      visualize=False,
update=False,      project=yolov5/runs/detect,      name=exp,
exist_ok=False,      line_thickness=3,      hide_labels=False,
hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 🚀 v7.0-398-g5cdad892 Python-3.11.11 torch-2.5.1+cu124
CUDA:0 (Tesla T4, 15095MiB)

Fusing layers...
Model summary: 157 layers, 7015519 parameters, 0 gradients, 15.8
GFLOPs
image 1/1 /content/drive/MyDrive/FIAP/FIAP_NEXT/2025/test/Cópia
de foto (70).jpg: 640x480 1 Pessoa, 29.0ms
Speed: 0.7ms pre-process, 29.0ms inference, 128.1ms NMS per
image at shape (1, 3, 640, 640)
Results saved to yolov5/runs/detect/exp4
```

Código-fonte 11 - Resultado do teste do modelo
Fonte: Elaborado pelo autor (2025)

A figura a seguir mostra que no teste, o boneco foi detectado como pessoa com 61% de acurácia.



Figura 35 - Detecção do boneco com 61% de acurácia
Fonte: Elaborado pelo autor (2025)

Esse processo é para mostrar como é importante dominar as etapas de treinamento, validação e teste. Caso esteja exausto, realize uma pausa e volte quando estiver recuperado, pois agora vamos cuidar do hardware que vai detectar imagens reais e transmitir via Wi-Fi para o seu computador, onde estará rodando o modelo best.pt que você acabou de treinar, validar e testar.

2.2.6 Montagem do hardware

Para ser possível gravar o ESP32, é necessário fazer as ligações eletrônicas apresentadas na figura “ESP32-CAM conectado à interface USB da fabricante FTDI”. Observe que são necessários quatro jumpers (fios) para conectar o módulo vermelho FTDI ao ESP32-CAM e +1 jumper (cinza na figura) para ativar o modo programação do ESP32-CAM e, assim, o desenvolvedor poderá programar o ESP32-CAM via software Arduino IDE.

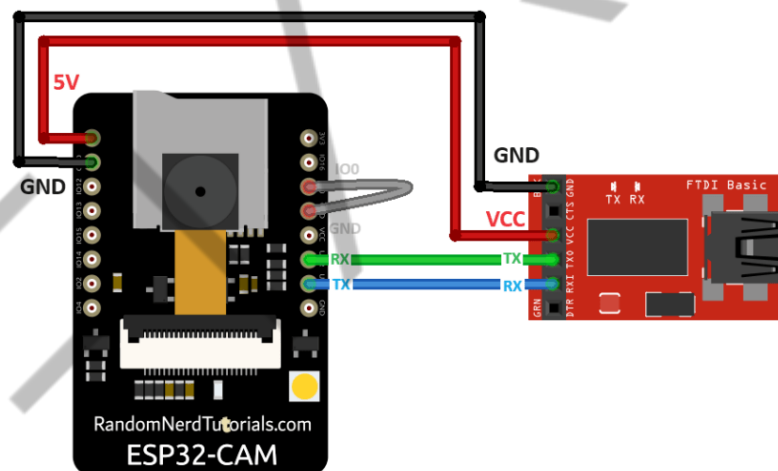


Figura 36 - ESP32-CAM conectado à interface USB da fabricante FTDI
Fonte: Random Nerd Tutorials (2020)

Detalhando as ligações:

1. Um fio de qualquer cor, mas de preferência preto, você liga o pino GND do ESP32 ao GND do FTDI;
2. Um fio de qualquer cor, mas de preferência vermelho, você liga o pino UOT do ESP32 ao pino TX do FTDI; um fio de qualquer cor, mas de preferência verde, você liga o pino RX do ESP32 ao pino RX do FTDI; um fio de qualquer cor, mas de preferência azul, você liga o pino TX do ESP32 ao pino RX do FTDI; e um fio de qualquer cor, mas de preferência cinza, você liga o pino UOT do ESP32 ao pino TX do FTDI;

ESP32: A Janela para a visão computacional

3. Um fio de qualquer cor, mas de preferência verde, você liga o pino UOT do ESP32-CAM ao pino RX do FTDI;
4. Um fio de qualquer cor, mas de preferência azul, você liga o pino UOR do ESP32-CAM ao pino TX do FTDI.
5. O quinto fio de qualquer cor, mas de preferência cinza, você só conecta quando for gravar o código-fonte no ESP32-CAM. A conexão é feita usando o 5º jumper entre os pinos GND e o IO1 da mesma plaquinha do ESP32-CAM. Vamos detalhar melhor esse 5º fio mais para frente. Por enquanto, não precisa conectá-lo.

A montagem real do circuito todo, composto de ESP32-CAM + placa interface FTDI + jumpers é apresentada nas próximas figuras. Note que o jumper cinza está solto porque será usado somente no momento de programar o chip.

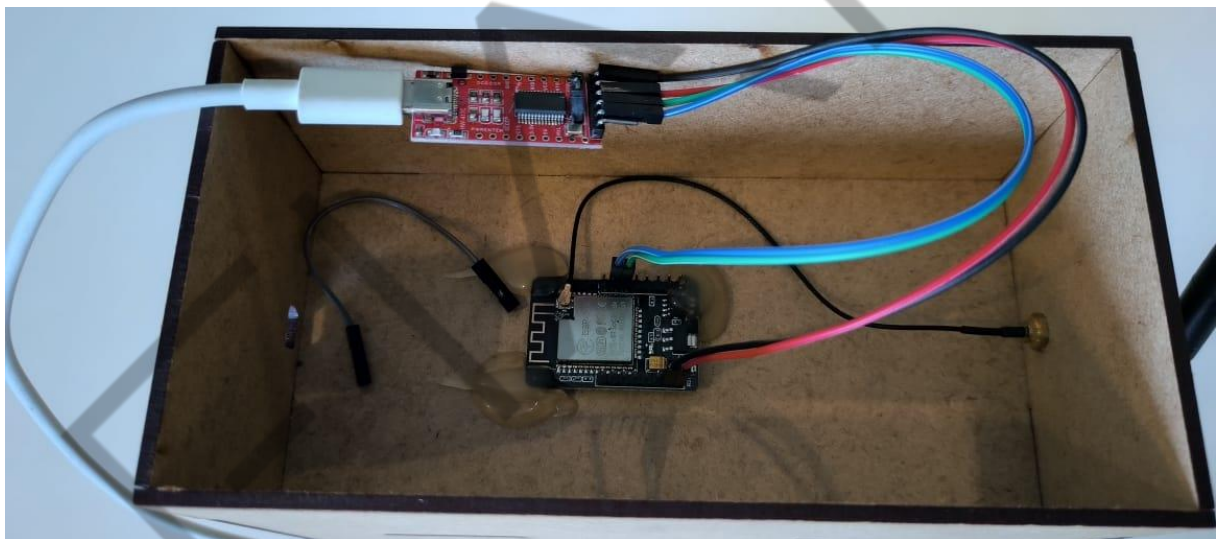


Figura 37 - Montagem real do ESP32-CAM
Fonte: Elaborado pelo autor (2025)

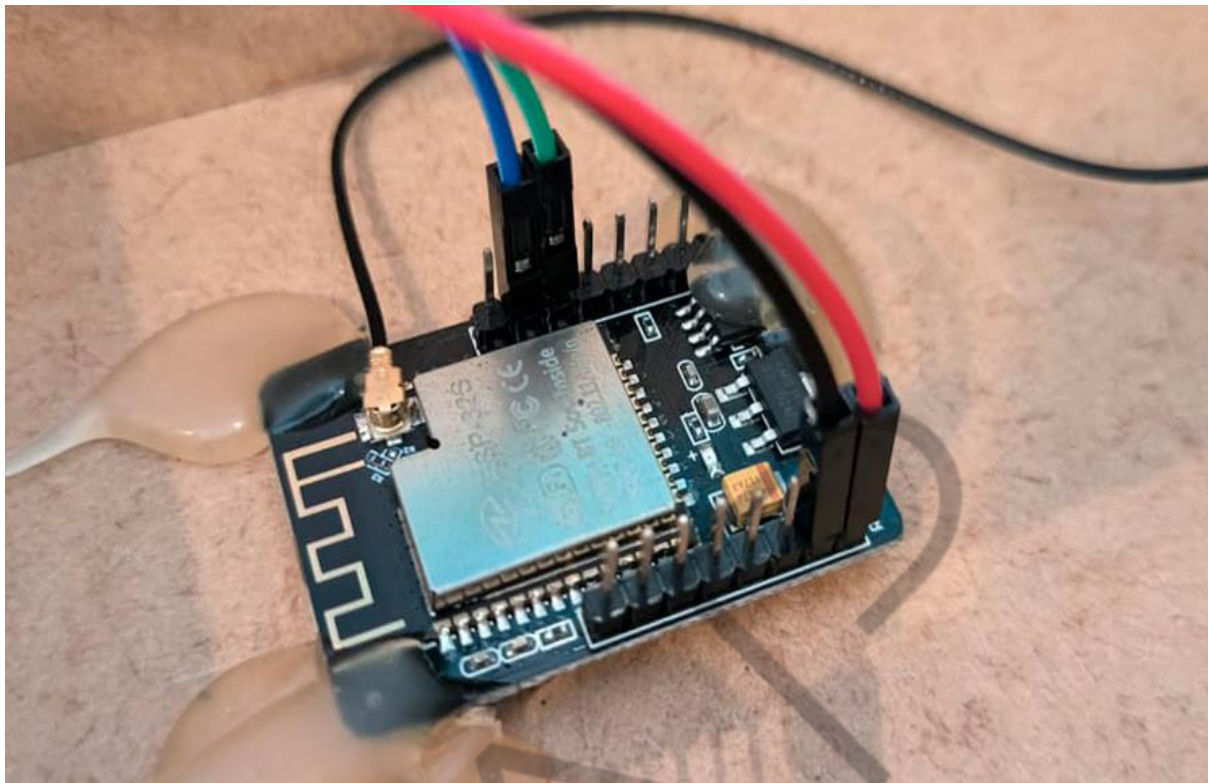


Figura 38 - Ligações eletrônicas detalhadas
Fonte: Elaborado pelo autor (2025)

Agora, conecte a plaquinha na porta USB do seu computador. Seu sistema operacional deve reagir com um som ou solicitação de instalação de driver. Caso nada aconteça, digite “Gerenciador de Dispositivos” no campo “Pesquisar” do Windows, e entre nessa tela.

Dentro do Gerenciador, expanda a opção “Portas” e busque pela opção “USB Serial Port” ou “Silicon Labs”. Na frente deve aparecer um número, que no exemplo ilustrado na figura a seguir, está COM6. Isso é uma prova que seu sistema operacional conseguiu encontrar o adaptador FTDI.

ESP32: A Janela para a visão computacional

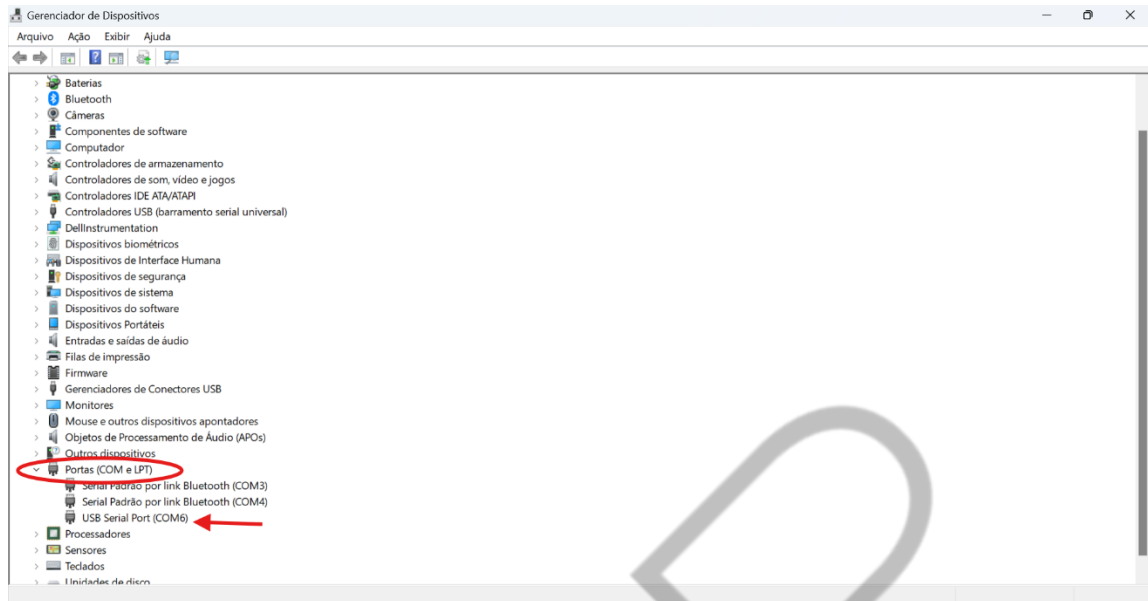


Figura 39 - Gerenciador de Dispositivos indicando o #USB da sua plaquinha
Fonte: Elaborado pelo autor (2025)

Caso não esteja vendo “Serial Port” ou “Silicon Labs”, baixe o driver mais atual presente no site do fabricante do chip USB que vai no FTDI, de acordo com o seu sistema operacional: <<https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>>.

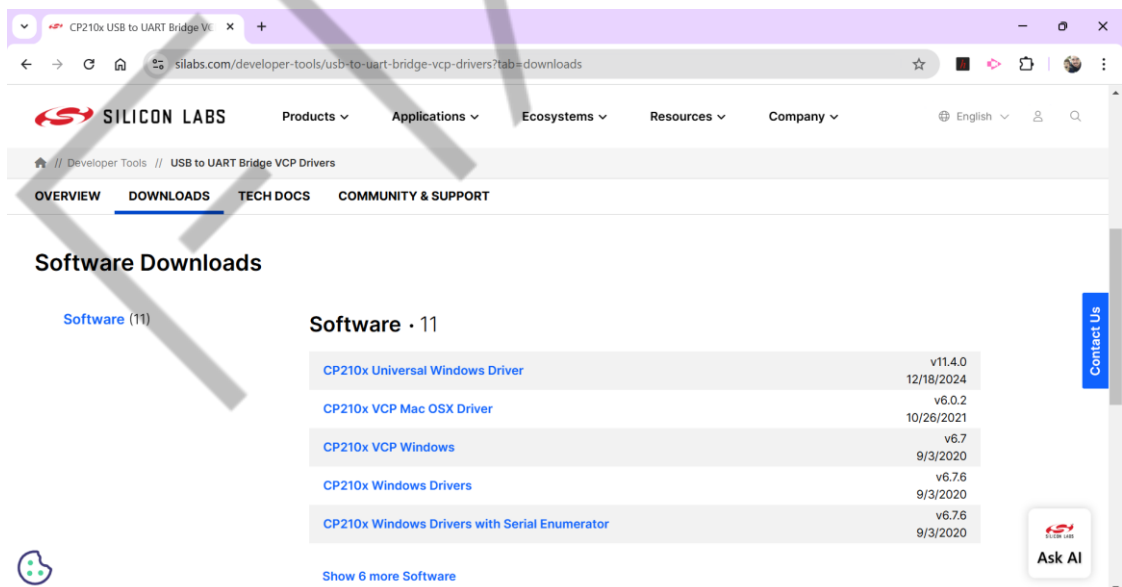


Figura 40 - Site da Silicon Labs, fabricante do chip USB disponível na placa FTDI
Fonte: Silabs (2025)

ESP32: A Janela para a visão computacional

Após baixar e instalar o driver CP210x em seu sistema, remova o cabo USB da sua plaquinha, reconecte-o no seu PC, e o Windows deverá reagir. Caso ainda não, reinicie seu Windows.

Agora, abra o seu Arduino IDE, copie o próximo código-fonte e cole na área indicada pela moldura vermelha da figura “Tela do Arduino IDE indicando onde colar o código-fonte fornecido”. Mas antes, **apague o void loop() e void setup() que estão no campo em vermelho.**

```
#include <WebServer.h>
#include <WiFi.h>
#include <esp32cam.h>

const char* WIFI_SSID = "ssid wifi residência ou celular";
//celular Apple não funciona, porque ele não tem 2.4GHz
const char* WIFI_PASS = "senha";
int8_t nivelBrilho = 100; //esse número indica o brilho do
flash. Varia de 0 (desligado) a 255 (brilho máximo). Não use
níveis acima de 200 porque vai aquecer o circuito do flash e
posteriormente, queimá-lo
#define FLASH_GPIO_NUM 4

WebServer server(80);

static auto loRes = esp32cam::Resolution::find(320, 240);
static auto midRes = esp32cam::Resolution::find(350, 530);
static auto hiRes = esp32cam::Resolution::find(800, 600);
void serveJpg()
{
    auto frame = esp32cam::capture();
    if (frame == nullptr) {
        Serial.println("CAPTURE FAIL");
        server.send(503, "", "");
        return;
    }
    Serial.printf("CAPTURE OK %dx%d %db\n", frame-
>getWidth(), frame->getHeight(),
                static_cast<int>(frame->size()));

    server.setContentLength(frame->size());
    server.send(200, "image/jpeg");
    WiFiClient client = server.client();
    frame->writeTo(client);
}

void handleJpgLo()
{
```

```
        if (!esp32cam::Camera.changeResolution(loRes)) {
            Serial.println("SET-LO-RES FAIL");
        }
        serveJpg();
    }

void handleJpgHi()
{
    if (!esp32cam::Camera.changeResolution(hiRes)) {
        Serial.println("SET-HI-RES FAIL");
    }
    serveJpg();
}

void handleJpgMid()
{
    if (!esp32cam::Camera.changeResolution(midRes)) {
        Serial.println("SET-MID-RES FAIL");
    }
    serveJpg();
}

void setup(){
    Serial.begin(115200);
    Serial.println();
    {
        using namespace esp32cam;
        Config cfg;
        cfg.setPins(pins::AiThinker);
        cfg.setResolution(hiRes);
        cfg.setBufferCount(2);
        cfg.setJpeg(80);

        bool ok = Camera.begin(cfg);
        Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL");
    }
    WiFi.persistent(false);
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.print("http://");
    Serial.println(WiFi.localIP());
    Serial.println("  /cam-lo.jpg");
    Serial.println("  /cam-hi.jpg");
    Serial.println("  /cam-mid.jpg");

    server.on("/cam-lo.jpg", handleJpgLo);
```

ESP32: A Janela para a visão computacional

```
server.on("/cam-hi.jpg", handleJpgHi);  
server.on("/cam-mid.jpg", handleJpgMid);  
  
server.begin();  
pinMode(FLASH_GPIO_NUM, OUTPUT);  
}  
  
void loop()  
{  
  server.handleClient();  
  analogWrite(FLASH_GPIO_NUM,nivelBrilho); //se o esp se  
  conectar no wifi, o flash acende fraquinho para não queimar.  
}
```

Código-fonte 12 - Código-fonte a ser gravado no ESP32-CAM

Fonte: Elaborado pelo autor (2025)

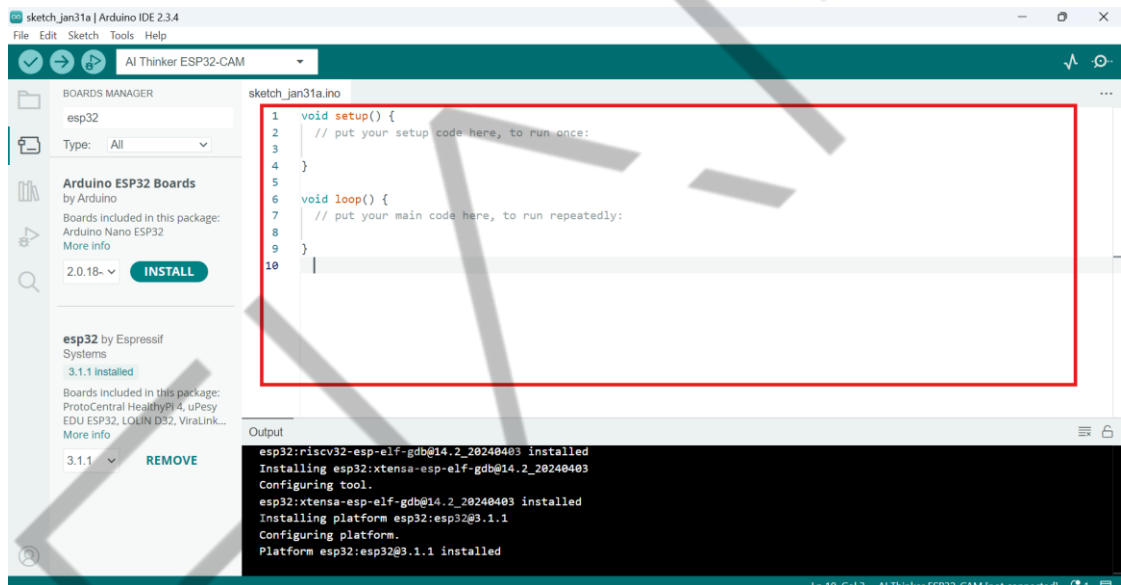


Figura 41 - Tela do Arduino IDE indicando onde colar o código-fonte fornecido

Fonte: Elaborado pelo autor (2025)

Após colar o código-fonte no campo indicado em vermelho, vamos instalar as bibliotecas necessárias para esse código-fonte ser compilado.

Acesse esse repositório do GitHub: <<https://github.com/espressif/esp32-camera>> e faça o download do zip para a seguinte pasta do seu computador: C:\Users\SeuNome\Documentos\Arduino\libraries, isto é, guarde o arquivo ZIP na subpasta **libraries** que está dentro do **Arduino**, que por sua vez, está dentro de **Documentos**. Veja como isso ficaria observando a figura a seguir.

ESP32: A Janela para a visão computacional

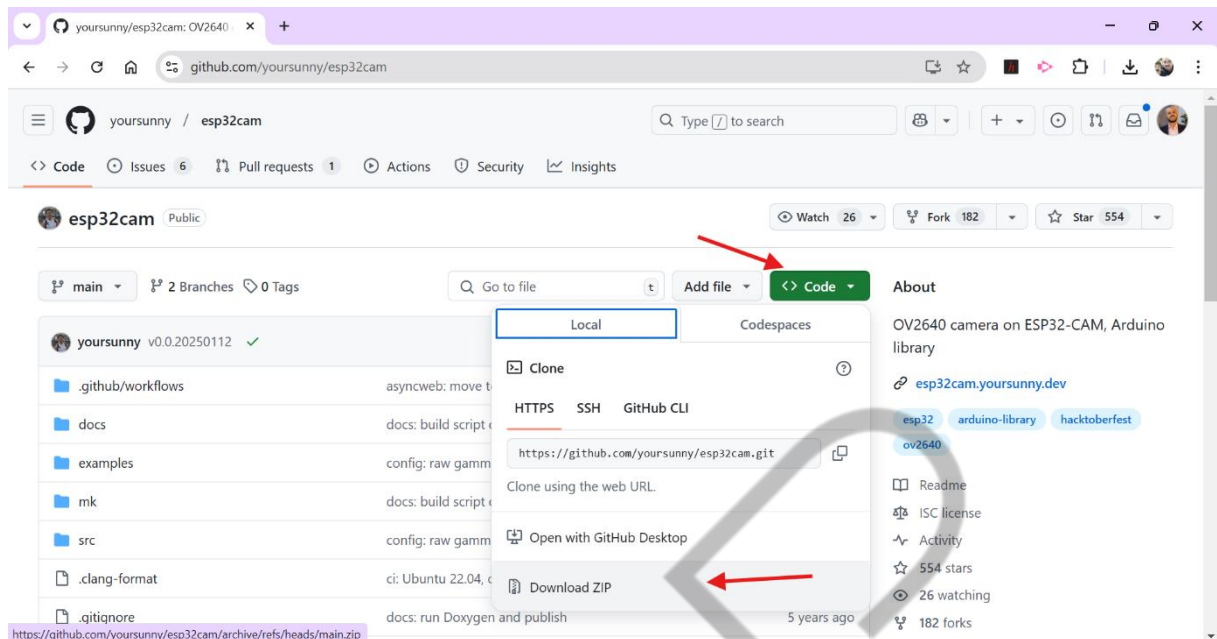


Figura 42 - Tela do repositório Git indicando como fazer download do *.zip
Fonte: Elaborado pelo autor (2025)

Agora, vamos adicionar essa biblioteca <<https://github.com/espressif/esp32-camera>> manualmente no Arduino IDE para que seja compilada juntamente com o nosso código do ESP32-CAM. Siga clicando nas setas da figura a seguir até chegar na pasta **C:\HOME\Documentos\Arduino\libraries** onde HOME deve ser o nome do seu PC. O objetivo aqui é selecionar o arquivo ZIP baixado.

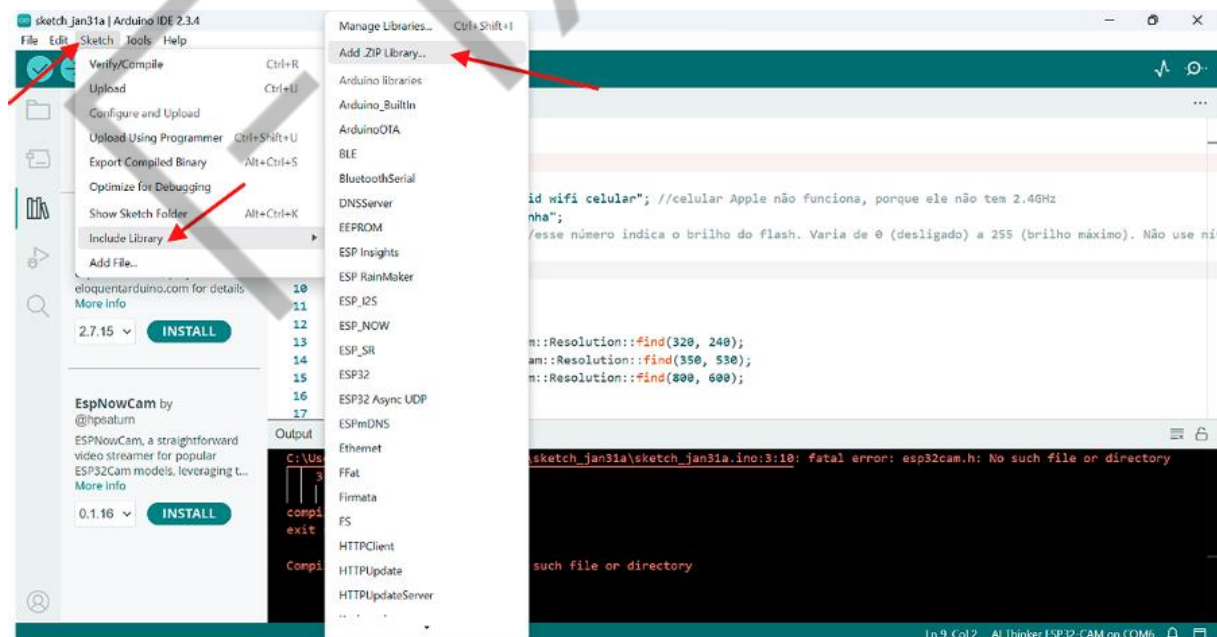


Figura 43 - Inclusão manual da biblioteca ZIP do GitHub
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

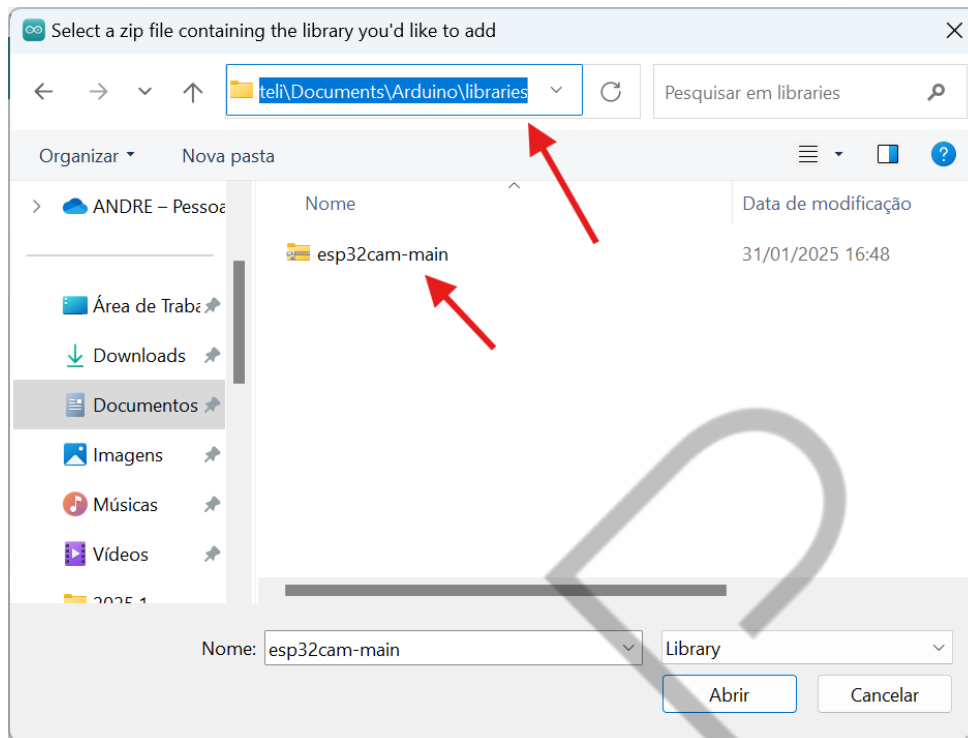


Figura 44 - Tela indicando o path e a biblioteca zip que acabou de chegar do GitHub
Fonte: Elaborado pelo autor (2025)

Após selecionado o arquivo ZIP, aperte “Abrir”, e aguarde a instalação dele. Isso leva 1 ou 2 segundos.

Agora, clique nos locais indicados pelas setas para você selecionar a porta USB que o seu PC atribuiu à sua placa ESP32-CAM. Essa definição de porta USB nem sempre é automática, e por isso, devemos fazê-la manualmente.

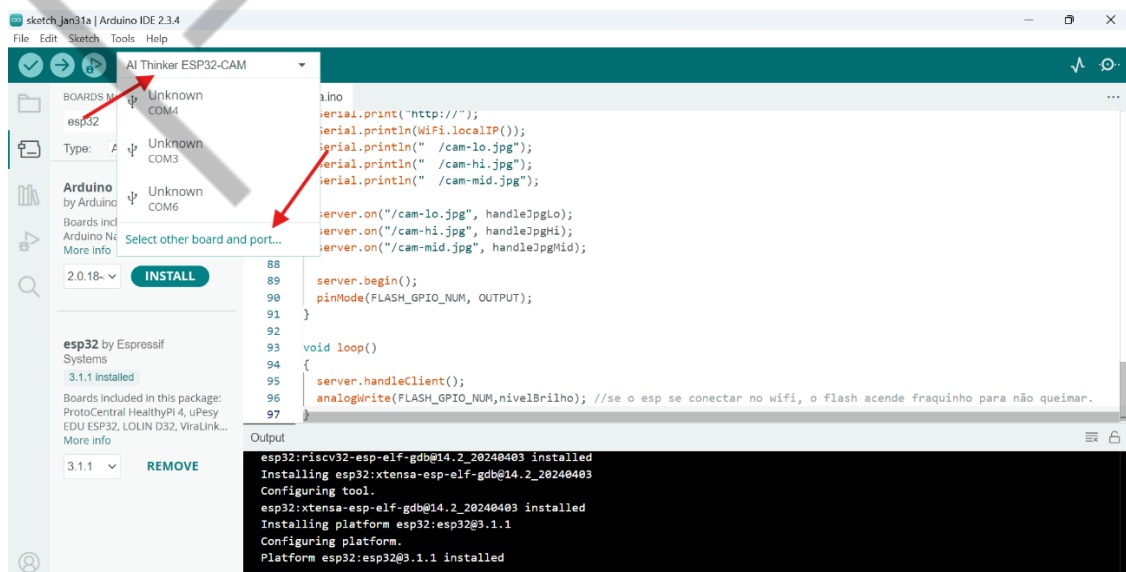


Figura 45 - Indicação de cliques para selecionar a porta USB
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Esse número é específico para cada computador, isto é, sua porta USB pode ter outro número diferente do print a seguir. No exemplo, está COM6.

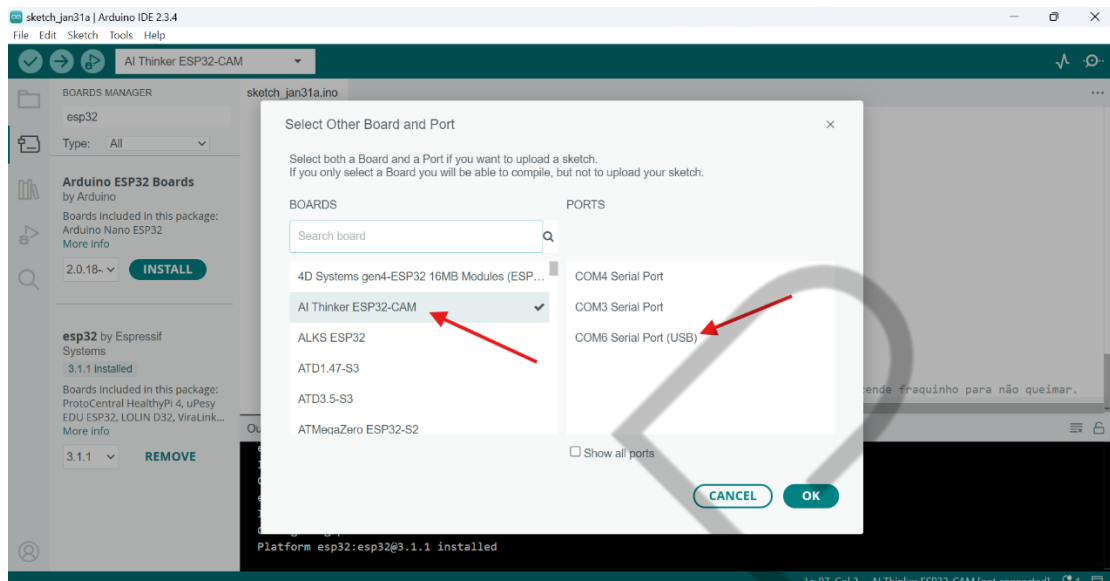


Figura 46 - Confira a seleção da placa AI Thinker e clique na porta que aparece “(USB)”

Fonte: Elaborado pelo autor (2025)

Agora, vamos conectar aquele 5º jumper para colocar o ESP32-CAM no modo programação. Esse jumper é essencial para essa etapa de gravação do código no chip.

Seguindo as figuras a seguir, conecte um jumper da cor de sua preferência entre os pinos GND e IO0. No caso da figura “Jumper cinza coloca o microcontrolador no modo “programação””, o jumper é o cinza. E na figura seguinte, também é cinza.

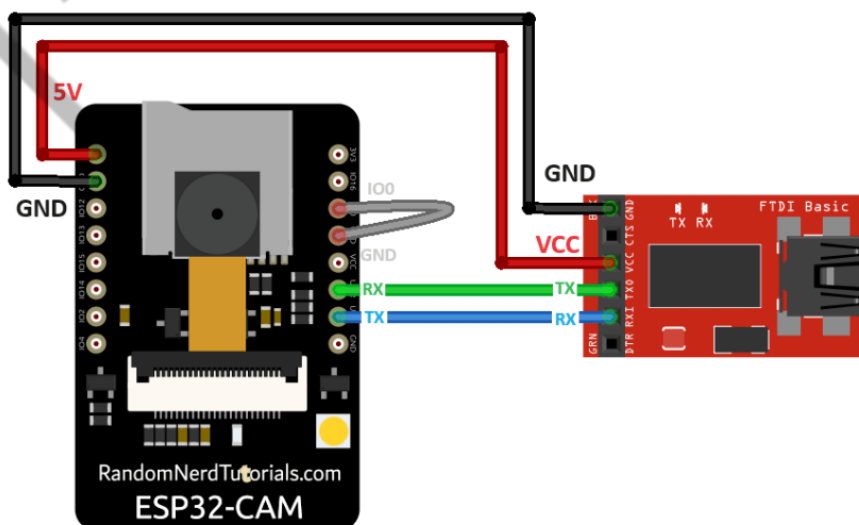


Figura 47 - Jumper cinza coloca o microcontrolador no modo “programação”

Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

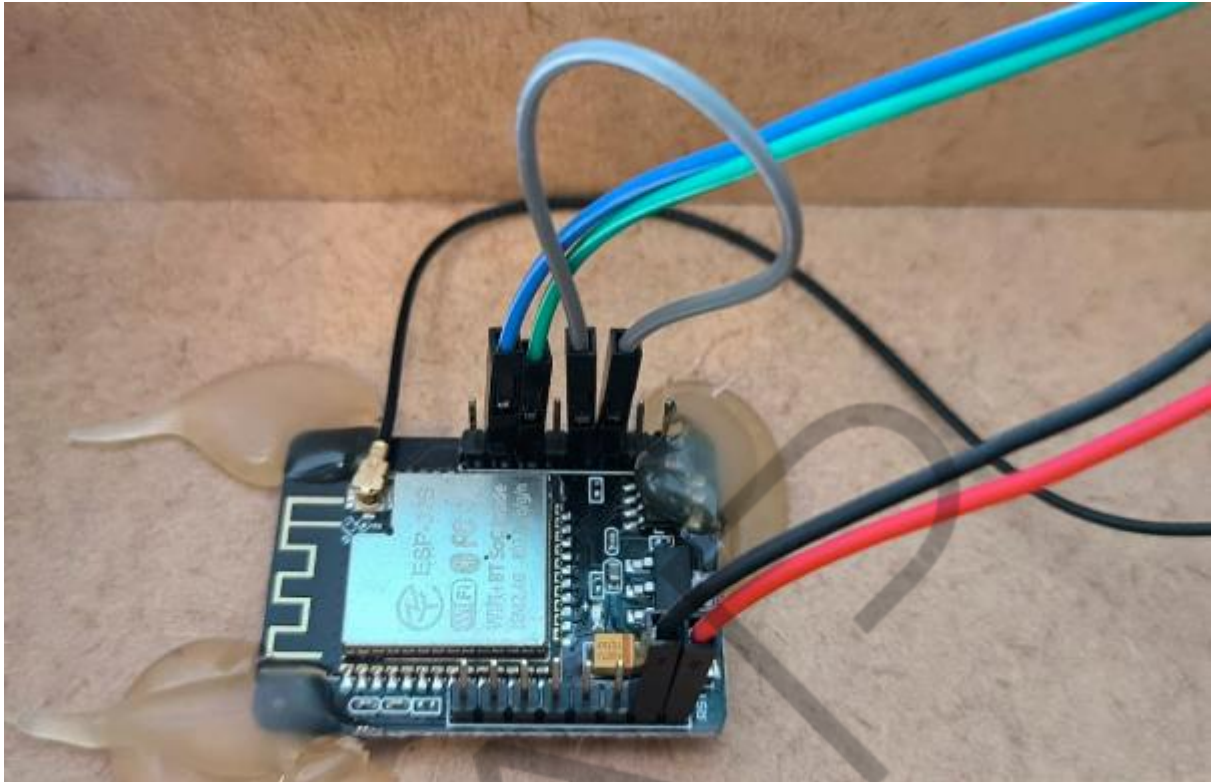


Figura 48 - Nessa foto real, jumper cinza acionando o modo “programação”
Fonte: Elaborado pelo autor (2025)

Agora, pressione 1x o botão “reset” que tem na plaquinha. Esse reset faz a plaquinha perceber que o jumper de “programação” foi colocado.

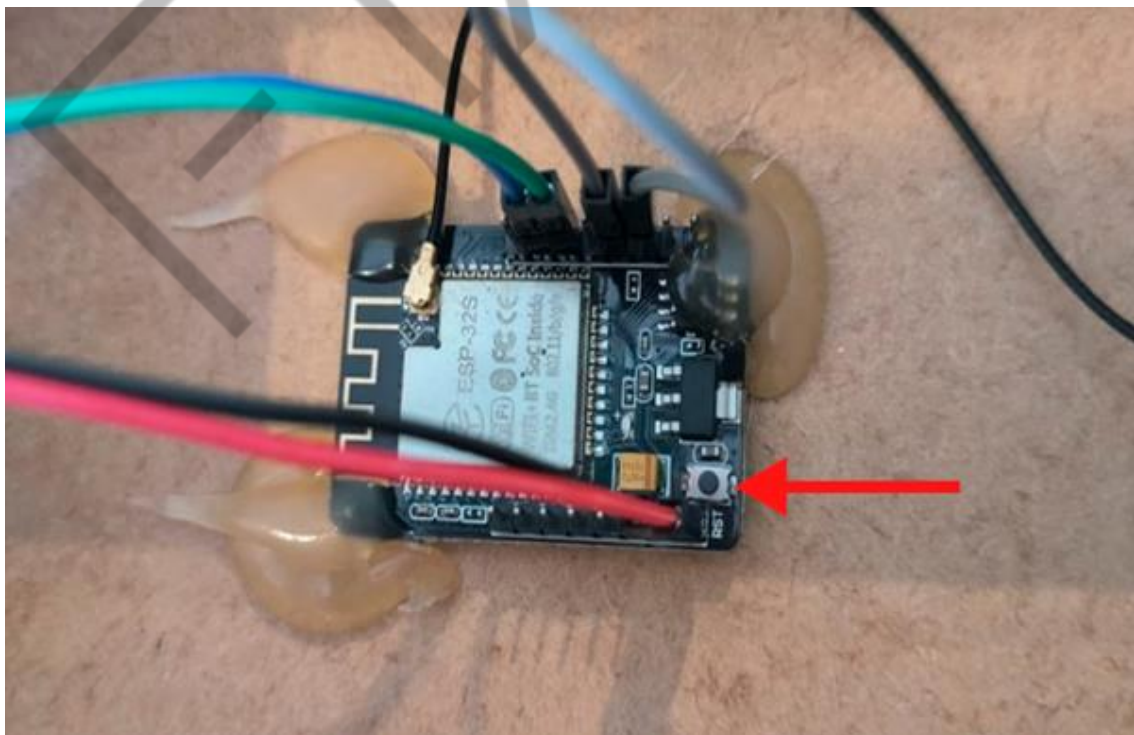


Figura 49 - Botão reset no detalhe da seta vermelha. Pressione 1x após encaixar o jumper cinza
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Feito, agora vamos gravar o código-fonte no ESP32-CAM. Certifique-se que colocou o nome do Wi-Fi e senha nas linhas 5 e 6. Lembre-se: sinais Wi-Fi que operam em 5GHz não são compatíveis com o ESP32-CAM, isso inclui o Wi-Fi roteado do iPhone.

Para gravar o código, siga a seta SUPERIOR da figura a seguir, e verifique a seta **inferior** se seu Ai Thinker ESP32-CAM está conectado numa COM.



Figura 50 - Clique no ícone “seta para direita” para gravar o ESP32-CAM
Fonte: Elaborado pelo autor (2025)

Quando começar a compilar, vai aparecer a mensagem indicada pela seta vermelha da figura “Código-fonte sendo compilado para depois ser gravado”.

ESP32: A Janela para a visão computacional

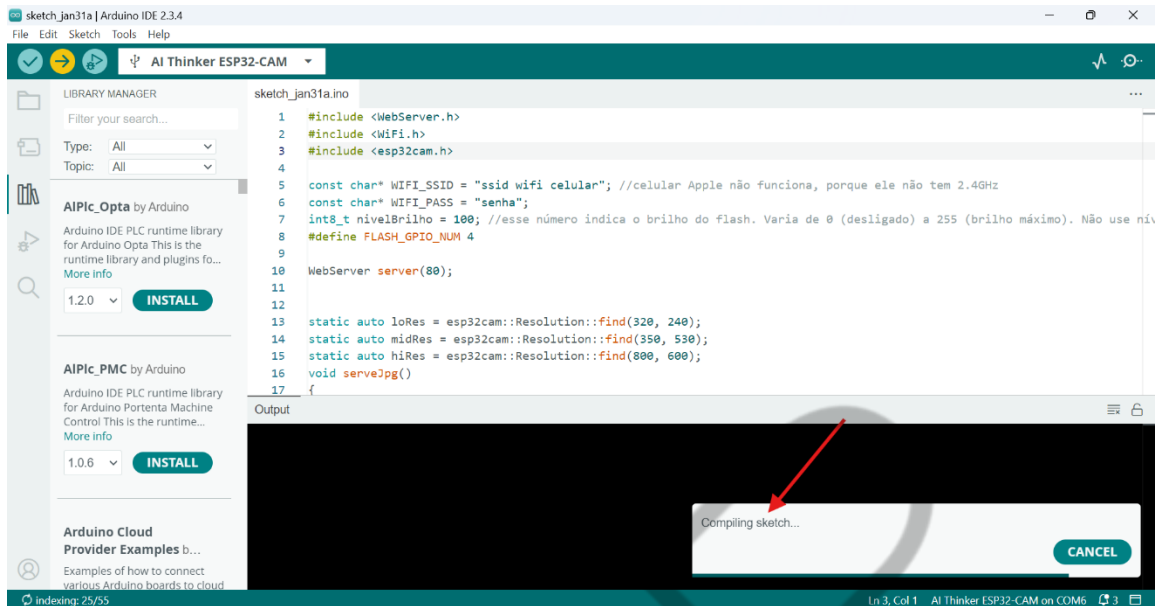


Figura 51 - Código-fonte sendo compilado para depois ser gravado
Fonte: Elaborado pelo autor (2025)

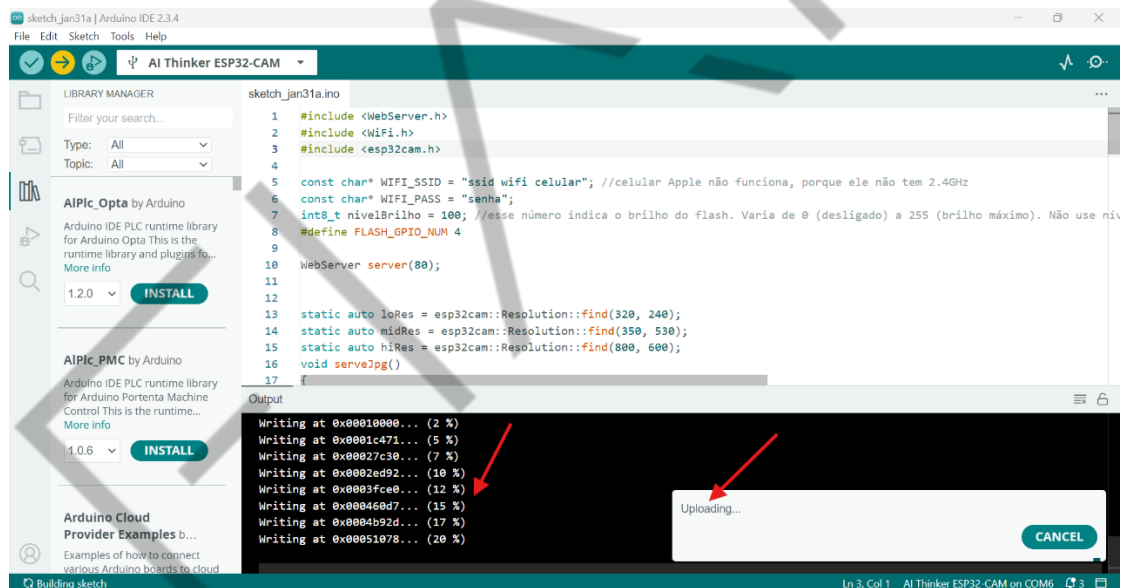


Figura 52 - Código-fonte sendo gravado no ESP32-CAM
Fonte: Elaborado pelo autor (2025)

A parte de configuração e programação do ESP32-CAM está finalizada. É importante lembrar de remover apenas o jumper de gravação da plaquinha e pressionar novamente o reset. Não precisa desligar a plaquinha da porta USB para remover o jumper de gravação.

Caso acenda um led branco na sua plaquinha ESP32-CAM, significa que deu tudo certo, seu ESP32 está conectado no Wi-Fi e transmitindo imagens.

ESP32: A Janela para a visão computacional

Agora você pode verificar as fotos que o seu ESP32-CAM está tirando. Para isso, descubra qual IP o seu ESP32-CAM pegou do DHCP (Dynamic Host Configuration Protocol) do seu roteador. Existem algumas maneiras para isso, e a mais simples é você clicar no **Monitor Serial** do Arduino IDE indicado pela seta que aponta para um ícone de lupa no canto direito superior da sua tela (ilustrado na figura “Indicação de como se abre o MONITOR SERIAL do Arduino IDE”).

Outras maneiras são: você entra no seu roteador e verifica a tabela de IP distribuída aos dispositivos conectados ou faz uma tabela para executar pings na faixa de IP da rede do Wi-Fi via Python para verificar em qual endereço ele responde esse ping.



Figura 53 - Indicação de como se abre o MONITOR SERIAL do Arduino IDE

Fonte: Elaborado pelo autor (2025)

O passo seguinte é alterar o *baud rate* já discutido em capítulos anteriores. O *baud rate* nativo do Monitor Serial é de 9600 bps. Precisamos mudar para 115200, conforme mostra a seta a seguir.

ESP32: A Janela para a visão computacional



Figura 54 - Indicação de como alterar o *baud rate* do Monitor Serial de 9600 para 115200
Fonte: Elaborado pelo autor (2025)

Agora, pressione novamente o botão reset da sua plaquinha que ela vai reiniciar o processo do DHCP e te informar qual IP ela pegou do seu Wi-Fi. No exemplo a seguir, o IP é 192.168.16.5.

Para você ver suas fotos em tempo real, abra um navegador qualquer e digite a URL: <IP/cam-hi.jpg>, isto é, <192.168.16.5/cam-hi.jpg>. Lembre-se de não incluir os sinais de menor/menor.

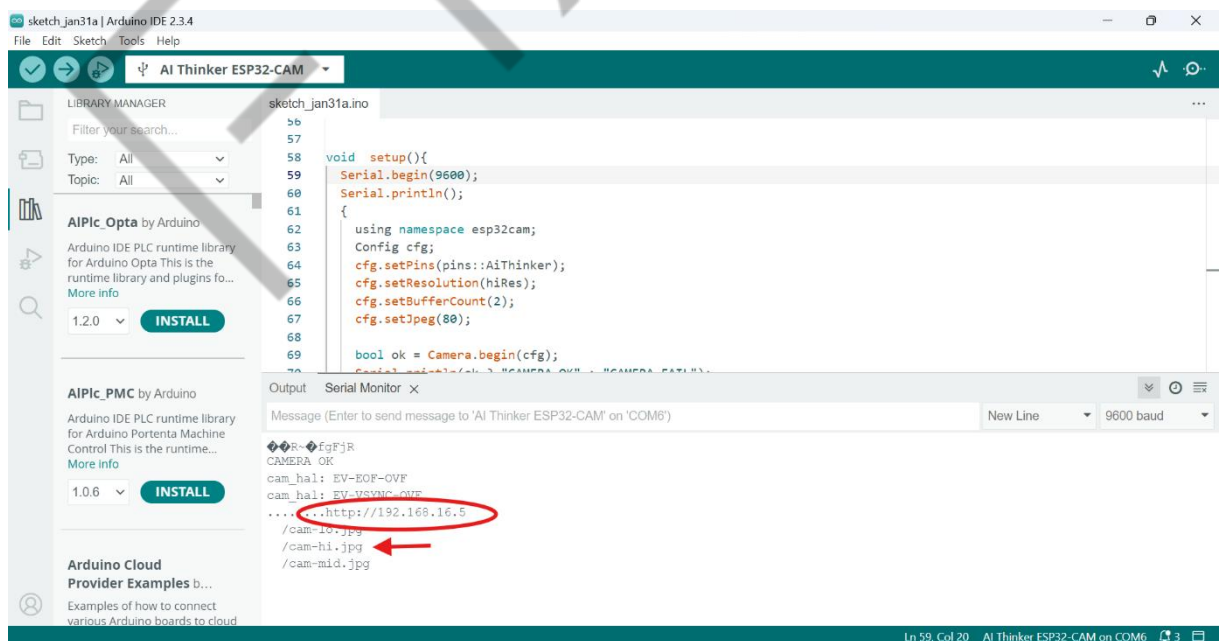


Figura 55 - Tela do Monitor Serial indicando qual IP chegou. No destaque, use /cam-hi.jpg
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Você pode escolher qual a qualidade da imagem prefere puxar do seu ESP32-CAM pela URL.

- /cam-hi.jpg → puxará a imagem de maior resolução (800 x 600 px);
- /cam-mid.jpg → puxará uma imagem média (350 x 530 px);
- /cam-lo.jpg → puxará uma imagem de baixa resolução (320 x 240 px).

É importante lembrar que **seu computador precisa estar na mesma rede Wi-Fi que o seu ESP32-CAM para que você possa acessar as fotos transmitidas**. Confira se seu IP 192.168.X.X está na mesma faixa do IP que aparece no Monitor Serial.

Observação 1: o sinal Wi-Fi precisa ser o de 2,4GHz, conhecido como 2G.

Observação 2: um LED branco na plaquinha se acende quando há sucesso na conexão com o Wi-Fi.

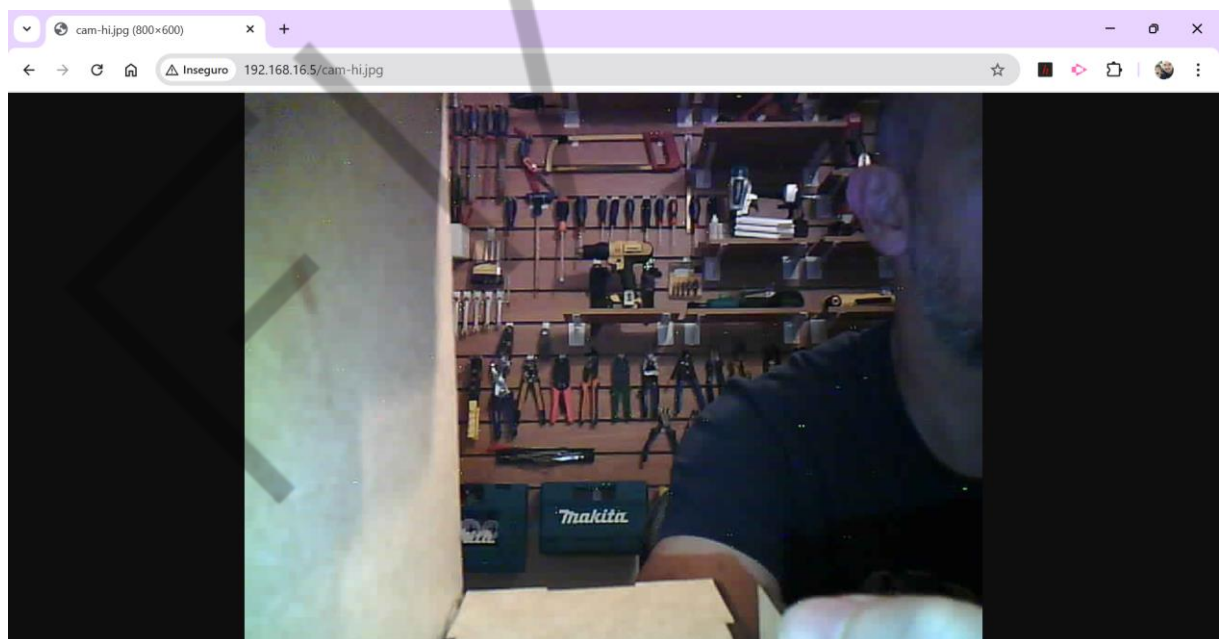


Figura 56 - Imagens sendo capturadas pelo ESP32-CAM e visualizadas no navegador Google Chrome

Fonte: Elaborado pelo autor (2025)

Alguns cuidados:

1. Sua plaquinha é muito sensível à curtos-circuitos. Então, jamais deixe o positivo e o negativo da plaquinha vermelha FDTI se encostarem. Evite objetos metálicos na sua mesa enquanto estiver manuseando a plaquinha;
2. Sua câmera também é sensível aos choques mecânicos. Sempre guarde seu kit na saboneteira após o uso. Depois de vários anos montando esse projeto, essa dica é de ouro!
3. E, por fim, todas as vezes que for gravar o ESP32-CAM, não se esqueça de adicionar o jumper de gravação e pressionar o reset, e para rodar o programa após ter gravado, remova o jumper e pressione novamente o reset para voltar ao modo “run”.

2.2.7 Integrando o ESP32-CAM com o modelo

Já sabemos que o ESP32-CAM não tem condições computacionais de rodar uma rede neural. Então a solução é transmitir via Wi-Fi para quem pode, que nesse caso, é o seu PC.

O código realiza quatro tarefas importantes:

1. Carrega o arquivo do modelo chamado **best.pt** que nasceu do processo de treinamento feito no Colab;
2. Conecta com o IP do seu ESP32-CAM via HTTP;
3. Coleta duas ou três imagens por segundo de tempo real via Wi-Fi;
4. Processa a imagem coletada e tenta reconhecer o objeto treinado no cenário.

Onde você vai rodar esse código-fonte a seguir? No VS Code que você instalou lá no começo desse capítulo!

Vamos aos passos: primeiro, abra o seu VS e, em seguida, crie um novo arquivo em Arquivo > Novo Arquivo. E quando te perguntar, diga que é um arquivo Python. Cole o seguinte código-fonte lá, e salve em um diretório de sua preferência.

```
import cv2
import torch
import numpy as np
import urllib
import pathlib
from pathlib import Path

pathlib.PosixPath = pathlib.WindowsPath

path = 'best.pt' # TROQUE PELO CAMINHO DO SEU PESO CASO QUEIRA
                 (best.pt que foi gerado no treinamento) ex:
                 yolov5/runs/train/exp9/weights/best.pt
image_url = 'http://192.168.10.250/cam-hi.jpg' # TROQUE PELO
LINK GERADO NO MONITOR SERIAL

model = torch.hub.load('ultralytics/yolov5', 'custom', path,
force_reload=True)
model.conf = 0.6 #essa variável determina a acurácia do seu
modelo. Faça testes para encontrar o melhor resultado.

print(path)

while True:
    img_resp=urllib.request.urlopen(url=image_url)
    imgnp=np.array(bytearray(img_resp.read()),dtype=np.uint8)
    im = cv2.imdecode(imgnp,-1)
    results = model(im)
    print(results)
    frame = np.squeeze(results.render())
    cv2.imshow('Deteccao', frame)
    key=cv2.waitKey(5)
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

Código-fonte 13 - Código que roda o modelo YOLO em tempo real integrado ao ESP32-CAM
Fonte: Elaborado pelo autor (2025)

Agora, pegue o arquivo best.pt no seu Colab e salve no mesmo diretório que você salvou o arquivo Python recente.

Onde encontrar o best.pt? Neste path: yolov5/runs/train/exp.../weights/best.pt

A figura a seguir mostra esse caminho:

ESP32: A Janela para a visão computacional

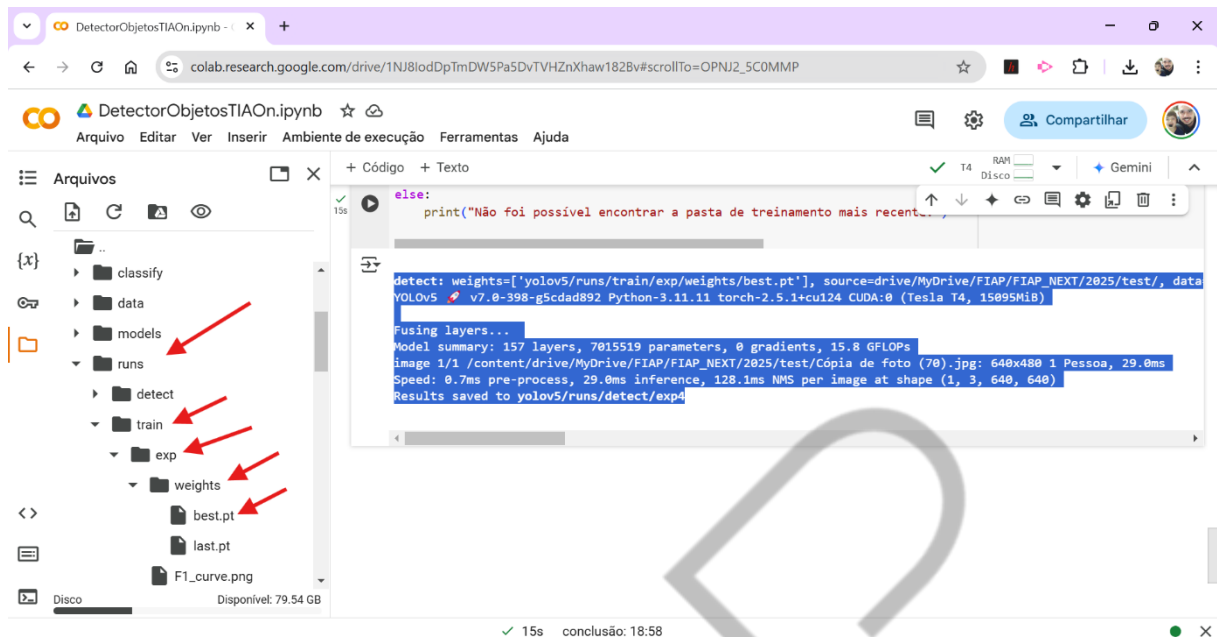


Figura 57 - Local de armazenamento do best.pt dentro do Colab
Fonte: Elaborado pelo autor (2025)

Clique nos três pontinhos que tem no best.pt, faça o download e salve-o no mesmo local onde está o seu arquivo Python recente. No caso desse projeto, o arquivo best.pt está com 14MB. Há casos de sucesso onde o modelo best.pt ficou com 1GB e um PC Core i7 conseguiu rodar tranquilamente.

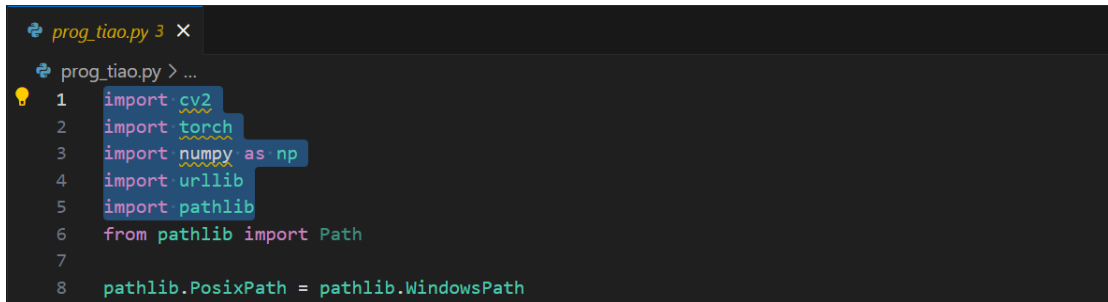
Agora, conecte sua placa FTDI no seu PC caso não tenha conectado ainda, abra o Arduino IDE caso não esteja aberto, abra o Monitor Serial e pegue o IP que aparece nele. Caso não tenha o IP já na tela, pressione o botão reset do ESP32-CAM para ele refazer a conexão e lhe informar o IP adotado.

Copie esse IP e atualize a linha 11 do código Python do VS Code. Abra um terminal do VS Code no path que está o arquivo *.py. Caso não tenha o pip install no seu VS Code, dê um:

```
python.exe -m pip install --upgrade pip
```

Código-fonte 14 - Instalando e atualizando o gerenciador Pip
Fonte: Elaborado pelo autor (2025)

Na sequência, vamos instalar os pacotes necessários para rodar o Python no VS:



```
prog_tiao.py 3 x
prog_tiao.py > ...
1 import cv2
2 import torch
3 import numpy as np
4 import urllib
5 import pathlib
6 from pathlib import Path
7
8 pathlib.PosixPath = pathlib.WindowsPath
```

Figura 58 - Precisamos instalar esses pacotes. Alguns serão inclusos em outros
Fonte: Elaborado pelo autor (2025)

```
pip install opencv-python
```

Código-fonte 15 - Instalando o opencv no seu VS Code
Fonte: Elaborado pelo autor (2025)

```
pip install torch
```

Código-fonte 16 - Instalando o Torch
Fonte: Elaborado pelo autor (2025)

Tenha paciência para aguardar o terminal do VS informar que finalizou as instalações ou que precisa de outro pip.

```
pip install pandas
```

Código-fonte 17 - Instalando o Pandas, gerenciador de banco de dados
Fonte: Elaborado pelo autor (2025)

```
pip install requests
```

Código-fonte 18 - Instalando o Requests
Fonte: Elaborado pelo autor (2025)

```
pip install pillow
```

Código-fonte 19 - Instalando o Pillow
Fonte: Elaborado pelo autor (2025)

Agora, finalmente dê um play “Run Python File” no canto direito superior, aguarde novas requisições serem satisfeitas. Caso dê um novo erro, é provável que seja a versão errada do Numpy. Então remova-o e instale-o novamente.

```
pip uninstall numpy scipy -y
```

Código-fonte 20 - Removendo versões incompatíveis do Numpy e do Scipy
Fonte: Elaborado pelo autor (2025)

```
pip install --no-cache-dir numpy==1.26.4 scipy==1.11.4
```

Código-fonte 21 - Instalando uma versão específica
Fonte: Elaborado pelo autor (2025)

ESP32: A Janela para a visão computacional

Se tudo ocorrer bem, uma tela pop-up chamada “Detecção” abrirá na sua frente, e o processo de reconhecimento dará início. A primeira vez que o modelo rodar, geralmente acontece um download de mais arquivos de forma automática. Porém, a partir da segunda vez, esse download não acontece mais.



Figura 59 - Teste final do reconhecedor usando uma segunda tela
Fonte: Elaborado pelo autor (2025)

3 APLICAÇÕES NO MUNDO REAL

Você deve estar se perguntando: “Por que um boneco preso no telhado?!” Porque essa maquete foi um projeto de 2024 apresentado no NEXT da FIAP, onde os alunos da turma de IA foram desafiados a criar um modelo de visão computacional que localizasse pessoas ilhadas no telhado numa enchente, pedindo socorro, isto é, um projeto inspirado para ajudar tragédias como a do Rio Grande do Sul que ocorreu naquele ano.

O ESP32-CAM foi afixado em um drone que sobrevoava a maquete do telhado com o boneco, representando uma pessoa. As imagens foram transmitidas para um notebook, cujo tinha todos os arquivos best.pt de cada grupo.

Os alunos da turma fazem uma competição em uma arena específica para drones dentro do evento. A regra do jogo é avaliar o modelo de melhor acurácia.

A parte de geolocalização do drone não foi integrada ao projeto, mas há grande potencial para isso.

No dia a dia, esse projeto pode lhe inspirar como uma primeira versão de visão computacional aplicável e desenvolvida por você ou pelo seu grupo. Por exemplo:

- **Agricultura inteligente (Agricultura 4.0):**
 - Detecção de pragas e doenças em plantas através da análise de imagens de folhas e frutos.
 - Monitoramento de colheitas via drones e satélites para otimização do uso de água e fertilizantes.
 - Classificação automatizada de grãos e frutas em indústrias agrícolas.
- **Segurança e vigilância:**
 - Reconhecimento facial para controle de acesso e autenticação.
 - Monitoramento inteligente de câmeras de segurança para detecção de atividades suspeitas.
 - Identificação de placas de veículos (LPR — License Plate Recognition) para controle de trânsito e pedágios.

- **Saúde e medicina:**

- Diagnóstico por imagem (radiografias, tomografias, ressonâncias) para detecção de doenças como câncer.
- Monitoramento de pacientes usando análise de vídeo para prever quedas ou emergências.

- **Indústria e manufatura:**

- Inspeção de qualidade automatizada, identificando falhas em produtos ou componentes.
- Controle robótico de linhas de produção, permitindo que máquinas ajustem processos em tempo real.
- Leitura de códigos de barras e QR Codes para rastreamento de produtos e logística inteligente.

CONCLUSÃO

A aplicação do ESP32-CAM em visão computacional demonstra como é possível utilizar dispositivos de baixo custo em projetos avançados de IA utilizando redes neurais. A combinação desse microcontrolador com o modelo YOLO possibilita a detecção com boa eficiência de objetos, abrindo caminho para diversas aplicações em agricultura, segurança, automação e monitoramento inteligente.

O processo de desenvolvimento do projeto envolve várias etapas fundamentais, desde a coleta e organização de imagens até o teste final da detecção.

A escolha de um conjunto de imagens bem estruturado e diversificado é essencial para garantir que o modelo tenha um alto grau de generalização.

Além disso, a etapa de pré-processamento dos dados no Make Sense AI é crucial para melhorar a qualidade da inferência e minimizar falhas no reconhecimento dos objetos.

A utilização do Google Colab para o treinamento do YOLOv5 permite a realização de experimentos computacionais mais robustos, aproveitando recursos gratuitos de GPU na nuvem.

A fase de validação e testes do modelo garante que o sistema seja confiável antes da implementação no hardware. Durante essa etapa, são ajustados hiperparâmetros e analisadas métricas como precisão (precision), recall e mAP (mean Average Precision) para otimizar o desempenho do modelo.

No que se refere ao hardware, a configuração do ESP32-CAM exige um ambiente de desenvolvimento adequado, como a Arduino IDE, além da implementação de uma comunicação HTTP eficiente com o servidor (seu PC local) onde o modelo YOLOv5 processa as imagens.

Apesar de suas vantagens, o ESP32-CAM apresenta algumas limitações de hardware, como a baixa memória RAM e o processamento restrito para inferência local de modelos complexos.

Para superar essas restrições, é recomendado delegar o processamento das imagens para servidores mais potentes, como o seu computador, e otimizar o envio dos dados por meio de compressão e filtros inteligentes. Estratégias como uso de

modelos mais leves e integração com serviços na nuvem podem aumentar a eficiência do sistema.

Esse capítulo evidencia a viabilidade de se utilizar ESP32-CAM com YOLOv5 para aplicações reais de visão computacional. Com uma abordagem bem planejada, é possível desenvolver sistemas inteligentes de baixo custo, otimizando a detecção de objetos em diferentes cenários.



REFERÊNCIAS

CP210x USB to UART Bridge VCP Drivers. **Silabs**, 2025. Disponível em: <<https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>>. Acesso em: 06 fev. 2025.

GOMES, Juliana Freitas Santos; LETA, Fabiana Rodrigues. Applications of computer vision techniques in the agriculture and food industry: a review. **European Food Research and Technology**, v. 235, p. 989-1000, 2012.

HOW to Program / Upload Code to ESP32-CAM AI-Thinker (Arduino IDE). **Random Nerd Tutorials**, 2020. Disponível em: <<https://randomnerdtutorials.com/program-upload-code-esp32-cam/>>. Acesso em: 06 fev. 2025.

LISZEWSKI, Andrew. Novo Raspberry Pi 4 é basicamente um desktop que custa apenas US\$ 35. **Uol**, 24 de junho de 2019. Disponível em: <<https://gizmodo.uol.com.br/raspberry-pi-4-desktop/>>. Acesso em: 06 fev. 2025

MIRANDA, Davy Rodrigo de; BOZONI, João Vinícius; RALL, Ricardo. Protótipo automatizado de esteira separadora utilizando Arduino Uno. **Fatec**, 17 de novembro de 2023. Disponível em: <<https://www.fatecbarueri.edu.br/revista/index.php/ingetec/article/view/104>>. Acesso em: 06 fev. 2025.

NEMIRE, Brad. Getting Started on Jetson – Top Resources from GTC 21. **Nvidia Developer**, 04 de junho de 2021. Disponível em: <<https://developer.nvidia.com/blog/getting-started-on-jetson-top-resources-from-gtc-21/>>. Acesso em: 06 fev. 2025.

PANERU, Suman; JEELANI, Idris. Computer vision applications in construction: Current state, opportunities & challenges. **Automation in Construction**, v. 132, p. 103940, 2021.

SZELISKI, Richard. **Computer vision: algorithms and applications**. Springer Nature, 2022.

แดงปลาสวรรค์; 262 | มาเล่น ESP32-CAM กับ ESPHome กัน. **YouTube**, 07 de setembro de 2021. Disponível em: <<https://www.youtube.com/watch?v=2ahkz9qgcec>>. Acesso em: 06 fev. 2025.